

**Interpolação Nativa em Malha de Voronoi para o
Modelo MPAS-A:
Geração de Condições Inicial e de Fronteira Lateral
Sem Reamostragem em Grade Regular**

Relatório técnico-científico do desenvolvimento *selfgrib* / rota
voronoi-to-voronoi

Diego Pereira
Instituto Nacional de Pesquisas Espaciais (INPE)
`diego.pereira@inpe.br`

Setembro de 2026

Resumo

Este relatório documenta o desenvolvimento de uma rota alternativa de geração de condições inicial (`init.nc`) e de fronteira lateral (`lbc.*.nc`) para o núcleo de previsão MPAS-A (*Model for Prediction Across Scales – Atmosphere*) que interpola diretamente entre duas malhas de Voronoi esféricas – a malha nativa de uma rodada global já concluída e a malha regional de destino – sem nenhuma etapa intermediária de reamostragem em grade latitude-longitude regular. O trabalho parte do *selfgrib*, um *ungrib* alternativo já existente que usa a saída nativa de uma rodada global do próprio MPAS-A como fonte de dado meteorológico, eliminando a dependência de GRIB externo (GFS, BAM, Eta). A rota descrita aqui vai um passo além: em vez de reescrever o formato binário intermediário do WPS e deixar o `init_atmosphere_model` original reinterpoliar esse dado de volta para a malha nativa (usando bilinear/16-pontos sobre uma grade estruturada – o único modo de interpolação que esse programa aceita), o pipeline aqui apresentado reimplementa, em Fortran, extraindo literalmente as fórmulas do código-fonte de referência do MPAS-Model (versão 3.0.2), cada etapa necessária – interpolação horizontal baricêntrica no triângulo dual de Delaunay, geração da grade vertical terrain-following híbrida, interpolação vertical, balanço hidrostático, inicialização de campos de superfície/solo – escrevendo os arquivos `init.nc/lbc.*.nc` diretamente, prontos para alimentar o núcleo dinâmico `mpas_atmosphere` sem qualquer grade intermediária. A implementação foi validada campo a campo contra dados reais de produção (malha regional `SouthAmerica`, recorte de $x1.163842$, ~ 60 km de resolução) e, por fim, contra o próprio consumidor final: uma previsão de 24 horas rodada com o `mpas_atmosphere` real a partir dos arquivos gerados por essa rota, sem erros e com campos fisicamente sãos, cuja divergência frente à rota de referência (via WPS) cresce de forma suave e limitada ao longo do tempo de integração – o padrão esperado de duas trajetórias vizinhas de um sistema caótico, não de um artefato numérico.

Palavras-chave: MPAS-A; malha de Voronoi; triangulação de Delaunay; interpolação baricêntrica; coordenada vertical *terrain-following*; condição inicial; fronteira lateral; previsão numérica do tempo.

Sumário

1	Introdução	6
1.1	Motivação	6
1.2	Pergunta de pesquisa e escopo do trabalho	7
1.3	Contribuições	7
1.4	Organização do relatório	8
2	Fundamentação teórica	9
2.1	A malha do MPAS-A: tesselação de Voronoi centroidal esférica	9
2.2	Dualidade com a triangulação de Delaunay	10
2.3	Interpolação baricêntrica no triângulo dual	10
2.4	Coordenada vertical <i>terrain-following</i> híbrida	11
2.5	Balanço hidrostático de referência	12
3	Revisão da literatura	13
3.1	O núcleo dinâmico MPAS-A e a malha SCVT	13
3.2	Geração e otimização de malhas de Voronoi esféricas	14
3.3	Interpolação sobre malhas de Voronoi/Delaunay em outros domínios	14
3.4	Síntese	15
4	Arquitetura da rota de interpolação nativa	16
4.1	A rota original e sua limitação estrutural	16
4.2	A rota nativa: visão geral	16
4.3	Decisão de escopo: por que não um subconjunto	18
4.4	Generalização: leitura do <i>namelist</i> real em tempo de execução	19
5	Fase 1 – Interpolação horizontal nativa	20
5.1	Objetivo e reaproveitamento de código	20
5.2	Procedimento	21
5.3	Reconstrução do vento nas arestas	21
5.4	Validação numérica	22

6	Fase 2 – Grade vertical nativa	23
6.1	Objetivo	23
6.2	Perfil vertical de referência $z_w(k)$	23
6.3	Coordenada híbrida e o achado do bloqueio	24
6.4	Suavização do terreno e da grade vertical	25
6.5	Campos derivados e coeficientes numéricos	26
6.5.1	Termos de métrica para advecção de terceira ordem (zb, zb^3)	26
6.6	Validação numérica	27
7	Fase 3 – Interpolação vertical	28
7.1	Objetivo e achado sobre a rotina real	28
7.2	A função de interpolação	28
7.3	Campos interpolados e particularidades por campo	29
7.4	Interpolação do vento	29
8	Fase 4 – Balanço hidrostático	30
8.1	Visão geral	30
8.2	Diagnóstico inicial de exner, θ e ρ_{zz}	30
8.3	Estado de referência isotérmico seco	31
8.4	Duas fórmulas de ρ_{zz} : por que não são redundantes	31
8.5	Solução hidrostática iterativa	32
8.6	Vento normal ponderado por densidade e vento vertical diagnóstico	32
8.7	Pressão de superfície diagnóstica e campos finais	33
9	Fase 6 – Campos de superfície e solo	34
9.1	Objetivo e achado de organização do código-fonte	34
9.2	Máscara terra/água e correção de temperatura de pele	34
9.3	Interpolação climatológica mensal de vegetação e albedo	35
9.4	Neve, gelo marinho e umidade de solo	35
10	Fase 7 – Escrita dos arquivos e execução do modelo	37
10.1	Composição do <code>init.nc</code> completo	37
10.2	Dois bugs de formato encontrados apenas na execução real	38
10.3	Execução completa	39
11	Implementação em Fortran	40
11.1	Princípio norteador: extração literal, não reformulação	40
11.2	Estrutura de módulos	40
11.3	Convenção de precisão numérica: uma exceção deliberada	41
11.4	Leitura do <i>namelist</i> real em tempo de execução	41
11.5	Reúso de dependências e organização do build	43

12 Resultados	44
12.1 Validação da interpolação horizontal (Fase 1)	44
12.2 Validação do <code>init.nc</code> completo	45
12.3 Validação do <code>lbc*.nc</code>	45
12.4 Validação funcional: execução do modelo e comparação de previsão	46
13 Discussão e limitações conhecidas	48
13.1 Mistura de terreno de fronteira não implementada	48
13.2 Reamostragem vertical de solo simplificada	48
13.3 Reclassificação de gelo marinho não implementada	49
13.4 Umidade específica via umidade relativa não implementada	49
13.5 Escopo de validação	49
13.6 Um interpolante linear por partes: limite de suavidade	50
13.7 Síntese das limitações	51
14 Conclusão	52
14.1 Síntese	52
14.2 Estado atual e alcance da validação	53
14.3 Trabalhos futuros	53
Referências	55

Lista de Figuras

2.1	Esquema da dualidade Voronoi–Delaunay usada na interpolação horizontal nativa. As células poligonais (azul) formam a malha primal de Voronoi; conectando os centros de três células mutuamente vizinhas obtém-se um triângulo do dual de Delaunay (laranja). Um ponto arbitrário $\mathbf{p}$ – tipicamente o centro de célula da malha de <i>destino</i> – é localizado dentro de um único triângulo de Delaunay da malha de <i>origem</i> , e o campo escalar nesse ponto é obtido por interpolação baricêntrica dos três valores nos vértices do triângulo (Eq. (2.4)).	10
4.1	Comparação entre a rota original (via formato binário WPS e grade lat-lon intermediária, acima) e a rota nativa <i>voronoi-to-voronoi</i> apresentada neste trabalho (abaixo). A rota nativa elimina toda etapa de reamostragem em grade regular: da malha de Voronoi global de origem à malha de Voronoi regional de destino, o dado nunca deixa de estar sobre uma malha não estruturada.	17
10.1	Composição do <code>init.nc</code> final: os 54 campos calculados pelos programas desta rota são mesclados, via <code>ncks -A</code> , com os 81 campos de cópia direta do arquivo de malha estática.	37

Lista de Tabelas

7.1	Campos interpolados verticalmente na Fase 3 e seu modo de extrapolação abaixo do topo do dado de origem.	29
11.1	Módulos Fortran novos desenvolvidos neste trabalho.	41
12.1	Erro absoluto campo a campo entre o <code>init.nc</code> gerado por esta rota e o <code>init.nc</code> real de produção (17064 células, malha <code>SouthAmerica</code> , mesmo tempo inicial). Compilado e executado no build/ambiente real de produção (cluster Jaci), não apenas em ambiente de desenvolvimento local.	45
12.2	Erro absoluto médio/máximo entre o <code>lbc.*.nc</code> gerado por esta rota e os arquivos reais de produção, nos cinco tempos de fronteira do experimento (intervalo de 6 h).	46
12.3	Erro absoluto médio/máximo entre a previsão gerada a partir dos arquivos desta rota e a previsão de referência, em três tempos de saída ao longo da integração de 24 h.	46
13.1	Síntese das simplificações/limitações conhecidas, seu impacto no caso validado, e a condição sob a qual se tornariam relevantes.	51

Capítulo 1

Introdução

1.1 Motivação

O MPAS-A (*Model for Prediction Across Scales – Atmosphere*) [14] é um modelo global/-regional de previsão numérica do tempo cujo núcleo dinâmico opera sobre uma malha horizontal não estruturada: uma tesselação de Voronoi centroidal esférica (SCVT), cujos polígonos (predominantemente hexágonos, com um pequeno número de pentágonos e heptágonos nos pontos de maior curvatura da geração da malha) definem os volumes de controle das variáveis de massa, enquanto as arestas entre células vizinhas carregam a componente normal do vento. Essa escolha de malha – em contraste com as grades latitude-longitude regulares tradicionalmente usadas em modelos globais mais antigos – evita a convergência de meridianos nos polos e permite refinamento local suave sem as discontinuidades de grades aninhadas.

O projeto no qual este trabalho se insere, *selfgrib*, nasceu de uma pergunta simples: é possível gerar a condição inicial e a fronteira lateral de uma rodada regional do MPAS-A usando, como fonte de dado meteorológico, a saída nativa de uma rodada global do *próprio* MPAS-A, sem depender de nenhum GRIB externo (GFS, BAM, Eta) e sem aplicar nenhum *patch* no código-fonte do MPAS-Model? A resposta, documentada em detalhe no relatório principal do projeto, foi sim: o *selfgrib* lê o `history.nc` de uma rodada global concluída, extrai e interpola os campos necessários, e escreve o formato binário intermediário do WPS – o mesmo formato que o *ungrib* geraria a partir de um GRIB real – permitindo que o `init_atmosphere_model` original consuma esse dado sem modificação alguma.

Esse desenho tem uma limitação estrutural, no entanto, que motivou diretamente o trabalho aqui descrito. O caminho ponta a ponta da rota original é:

malha nativa global → **grade lat-lon regular** → formato binário WPS → malha nativa regional

Duas etapas desse caminho envolvem reamostragem espacial redundante: a malha nativa global é primeiro remapeada para uma grade lat-lon regular (pelo utilitário `convert_mpas`,

da NCAR), e depois o `init_atmosphere_model` reinterpola essa grade de volta para a malha nativa *regional* de destino, usando interpolação bilinear ou de 16 pontos sobre índices de grade estruturada – o único modo de interpolação horizontal que esse programa aceita para o caso de dado real (`config_init_case=7/9`). O resultado é um percurso *malha* → *grade* → *malha*: cada etapa de reamostragem introduz um erro de interpolação de ida e volta, mesmo quando a grade intermediária é fina, simplesmente pelo fato de trocar de representação espacial duas vezes em vez de zero. Esse problema já havia se manifestado concretamente no projeto como uma classe de bug real e documentada (a grade lat-lon nominal, dimensionada pela elipse teórica do recorte regional, ficava sistematicamente menor que a extensão real da malha recortada, deixando células sem dado válido na fronteira – ver Chapter 13).

1.2 Pergunta de pesquisa e escopo do trabalho

A pergunta que orienta este trabalho é direta: *é possível eliminar inteiramente a grade lat-lon intermediária, interpolando diretamente entre a malha de Voronoi de origem e a malha de Voronoi de destino, e escrevendo os arquivos de entrada do núcleo dinâmico do MPAS-A (`init.nc/lbc.*.nc`) diretamente – sem passar pelo formato binário do WPS nem pelo `init_atmosphere_model` original?*

A investigação começou pela leitura do código-fonte real do `init_atmosphere_model` (rotina `mpas_init_atm_hinterp.F`) para verificar se essa reinterpolação final para a malha regional aceitaria, de alguma forma, uma lista de pontos dispersos em vez de uma grade estruturada. A resposta encontrada foi negativa: o programa é estruturalmente amarrado ao formato de grade (a interpolação bilinear opera sobre índices (i,j) de uma matriz regular). Isso significa que *qualquer* ganho de precisão de uma interpolação nativa malha-a-malha só é alcançável pulando esse programa inteiramente para a etapa de horizontal/vertical/balanco físico — não há um “escopo mínimo” que preserve o formato WPS e ainda elimine o percurso malha→grade→malha. Essa constatação (detalhada na Chapter 4) definiu o escopo do trabalho: reimplementar, em Fortran, cada etapa fisicamente necessária para produzir um `init.nc/lbc.*.nc` válido – não um subconjunto delas.

1.3 Contribuições

Este trabalho contribui com:

- (i) Um motor de interpolação horizontal nativa malha-a-malha, que reaproveita sem modificação o mecanismo de pesos baricêntricos do utilitário `convert_mpas` (originalmente exposto apenas para remapear para grades lat-lon), operando em modo de

- lista de pontos dispersos (*scatter*) sobre os centros de célula da malha de destino;
- (ii) A extração e tradução literal, para um conjunto de módulos Fortran independentes e testáveis, das rotinas do núcleo dinâmico do MPAS-Model (versão de referência 3.0.2) responsáveis pela geração da grade vertical *terrain-following* híbrida, interpolação vertical, balanço hidrostático não linear, e inicialização de campos de superfície e solo;
 - (iii) Um escritor de `init.nc/lbc.*.nc` que produz arquivos NetCDF binariamente compatíveis com o que o `mpas_atmosphere` espera, sem depender do `init_atmosphere_model` original para gerá-los;
 - (iv) Uma generalização dos programas resultantes para ler os parâmetros de configuração (`config_*`) do `namelist.init_atmosphere` real do experimento em tempo de execução, tornando a rota reproduzível para qualquer malha e caso de estudo, não apenas para o exemplo de validação usado neste relatório;
 - (v) Validação em duas camadas: (a) numérica, campo a campo, contra arquivos `init.nc/lbc.*.nc` reais de produção; e (b) funcional, executando o `mpas_atmosphere` real a partir dos arquivos gerados por essa rota e obtendo uma previsão de 24 horas fisicamente sã, sem erros.

1.4 Organização do relatório

O Chapter 2 apresenta a fundamentação teórica necessária – a estrutura da malha SCVT do MPAS-A, sua dualidade com a triangulação de Delaunay, e o princípio da interpolação baricêntrica usada como método de remapeamento horizontal. O Chapter 3 situa esse método na literatura mais ampla sobre malhas de Voronoi em geociências e computação científica. O Chapter 4 descreve a arquitetura geral da nova rota (o *pipeline*) e a decisão de implementação de escopo completo. Os capítulos 5 a 10 descrevem cada fase da implementação em detalhe, com as equações e algoritmos efetivamente usados. O Chapter 11 discute as escolhas de engenharia da implementação em Fortran. O Chapter 12 apresenta os resultados de validação numérica e a execução funcional do modelo. O Chapter 13 discute limitações conhecidas e simplificações deliberadas. O Chapter 14 encerra com as conclusões e sugestões de trabalho futuro.

Capítulo 2

Fundamentação teórica

2.1 A malha do MPAS-A: tesselação de Voronoi centroidal esférica

A malha horizontal do MPAS-A é uma *tesselação de Voronoi centroidal esférica* (*Spherical Centroidal Voronoi Tessellation*, SCVT) [14, 4, 15]. Dado um conjunto de pontos geradores $\{\mathbf{x}_i\}_{i=1}^N$ distribuídos sobre a esfera, o diagrama de Voronoi particiona a superfície em N células $\{V_i\}$, onde cada célula é o lugar geométrico dos pontos da esfera mais próximos do gerador $\mathbf{x}_i$ do que de qualquer outro gerador:

$$V_i = \left\{ \mathbf{x} \in \mathbb{S}^2 : d(\mathbf{x}, \mathbf{x}_i) \leq d(\mathbf{x}, \mathbf{x}_j) \ \forall j \neq i \right\}, \quad (2.1)$$

com $d(\cdot, \cdot)$ a distância geodésica sobre a esfera. A tesselação é dita *centroidal* quando cada gerador $\mathbf{x}_i$ coincide com o centroide (baricentro de massa) de sua própria célula V_i – uma condição de otimalidade obtida iterativamente pelo algoritmo de Lloyd, e que produz células com forma próxima de hexágonos regulares na maior parte do domínio, minimizando distorção de área e permitindo refinamento local suave da resolução horizontal sem as discontinuidades abruptas de grades aninhadas tradicionais.

No MPAS-A, as variáveis prognósticas de massa (densidade seca ρ , temperatura potencial θ , razões de mistura de água q) são armazenadas nos centros de célula $\mathbf{x}_i$ (geradores da tesselação), enquanto a componente *normal* do vento horizontal u é armazenada nas arestas entre células vizinhas – um posicionamento tipo *C-grid* (Arakawa C) generalizado para malha não estruturada. Essa convenção de estagiamento é o que motiva a separação, na interpolação horizontal nativa apresentada neste trabalho (Chapter 5), entre um tratamento para campos de massa (posicionados no centro da célula) e um tratamento posterior, distinto, para a reconstrução do vento nas arestas (Chapter 7).

2.2 Dualidade com a triangulação de Delaunay

Toda tesselação de Voronoi possui uma estrutura geométrica *dual*: a *triangulação de Delaunay*, obtida conectando por um segmento (ou, na esfera, um arco geodésico) cada par de geradores cujas células de Voronoi compartilham uma aresta. A propriedade central dessa dualidade, usada diretamente neste trabalho, é que os *vértices* da malha de Voronoi (pontos onde três ou mais células se encontram) são exatamente os *circuncentros* dos triângulos da triangulação de Delaunay – enquanto os *centros das células* de Voronoi (os geradores $\mathbf{x}_i$) são exatamente os *vértices* dos triângulos de Delaunay.

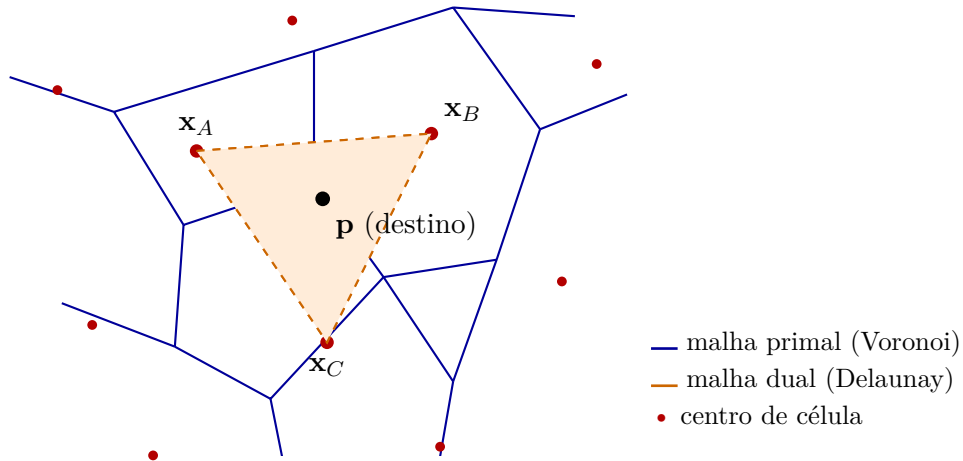


Figura 2.1: Esquema da dualidade Voronoi–Delaunay usada na interpolação horizontal nativa. As células poligonais (azul) formam a malha primal de Voronoi; conectando os centros de três células mutuamente vizinhas obtém-se um triângulo do dual de Delaunay (laranja). Um ponto arbitrário $\mathbf{p}$ – tipicamente o centro de célula da malha de *destino* – é localizado dentro de um único triângulo de Delaunay da malha de *origem*, e o campo escalar nesse ponto é obtido por interpolação baricêntrica dos três valores nos vértices do triângulo (Eq. (2.4)).

2.3 Interpolação baricêntrica no triângulo dual

Dado um triângulo de Delaunay com vértices $\mathbf{x}_A, \mathbf{x}_B, \mathbf{x}_C$ (centros de três células de Voronoi mutuamente vizinhas) e um ponto de interesse $\mathbf{p}$ localizado dentro desse triângulo, as *coordenadas baricêntricas* $(\lambda_A, \lambda_B, \lambda_C)$ de $\mathbf{p}$ são os únicos pesos que satisfazem simultaneamente

$$\mathbf{p} = \lambda_A \mathbf{x}_A + \lambda_B \mathbf{x}_B + \lambda_C \mathbf{x}_C, \quad \lambda_A + \lambda_B + \lambda_C = 1, \quad \lambda_A, \lambda_B, \lambda_C \geq 0. \quad (2.2)$$

Geometricamente, cada peso λ_A é proporcional à área do subtriângulo oposto ao vértice A (formado por $\mathbf{p}$, B e C), normalizada pela área total do triângulo ABC :

$$\lambda_A = \frac{\text{Área}(\mathbf{p}, B, C)}{\text{Área}(A, B, C)}, \quad \lambda_B = \frac{\text{Área}(A, \mathbf{p}, C)}{\text{Área}(A, B, C)}, \quad \lambda_C = \frac{\text{Área}(A, B, \mathbf{p})}{\text{Área}(A, B, C)}. \quad (2.3)$$

O valor interpolado de um campo escalar ϕ (por exemplo, temperatura, altura geopotencial, ou umidade específica) no ponto $\mathbf{p}$ é então a combinação linear

$$\phi(\mathbf{p}) \approx \lambda_A \phi(\mathbf{x}_A) + \lambda_B \phi(\mathbf{x}_B) + \lambda_C \phi(\mathbf{x}_C). \quad (2.4)$$

Esse método é exato para campos que variam linearmente sobre o triângulo e, criticamente para este trabalho, é *exato por construção* quando o ponto de interesse $\mathbf{p}$ coincide com um dos próprios vértices $\mathbf{x}_A$, $\mathbf{x}_B$ ou $\mathbf{x}_C$ (nesse caso um dos pesos vale 1 e os outros dois valem 0) – é essa propriedade que permite validar a implementação de forma direta: interpolando o campo de uma malha para si mesma (*auto-scatter*), o erro deve ser identicamente nulo em todas as células (Section 5.4).

Na esfera, o procedimento operacional usado pelo motor de pesos (herdado sem modificação do utilitário `convert_mpas` da NCAR, ver Section 5.1) é: (i) localizar, para cada ponto de destino $\mathbf{p}$, o triângulo de Delaunay da malha de origem que o contém, usando a conectividade célula-célula/aresta já armazenada na malha (`cellsOnCell`, `cellsOnVertex`); (ii) calcular os três pesos baricêntricos via Eq. (2.3), com as áreas computadas sobre a superfície esférica; e (iii) aplicar Eq. (2.4) a cada campo escalar requisitado. A reconstrução do vento, por armazenar sua componente normal nas arestas em vez de um vetor completo no centro de célula, exige um tratamento adicional – descrito em detalhe na Section 5.3 – baseado em Funções de Base Radial (RBF) para reconstrução vetorial nos centros de célula, seguida de projeção na direção normal de cada aresta de destino.

2.4 Coordenada vertical *terrain-following* híbrida

Na direção vertical, o MPAS-A usa uma coordenada ζ *terrain-following* (segue o relevo perto da superfície e tende a superfícies de altura constante longe dele), na formulação híbrida de Klemp [9]. A altura física z de uma superfície de coordenada constante $\zeta = \zeta_k$ é dada por

$$z(\zeta_k, \mathbf{x}) = \zeta_k + A(\zeta_k) h_s(\mathbf{x}, \zeta_k), \quad (2.5)$$

onde h_s é uma versão *suavizada* do terreno (progressivamente mais suave em níveis mais altos, para atenuar erros de gradiente de pressão horizontal sobre relevo íngreme – o problema clássico que motivou toda a família de coordenadas híbridas desde Gal-Chen e Simmons/Burridge até Klemp) e $A(\zeta)$ é uma função de atenuação que vale 1 na superfície

e decai suavemente a 0 numa altura de transição z_t , definindo o ponto acima do qual as superfícies de coordenada voltam a ser perfeitamente horizontais. As formas específicas de $A(\zeta)$ e do algoritmo de suavização de h_s efetivamente usadas pelo MPAS-A – e reproduzidas neste trabalho – são detalhadas com suas constantes exatas no Chapter 6.

2.5 Balanço hidrostático de referência

A condição inicial de um modelo não hidrostático como o MPAS-A ainda precisa de um estado *base* hidrostaticamente balanceado, em torno do qual as variáveis prognósticas são definidas como perturbações. O MPAS-A usa uma atmosfera de referência isotérmica seca ($T_0 = 250$ K), e o campo de pressão real é obtido resolvendo iterativamente a relação hidrostática discreta na direção vertical – não uma integração fechada, mas um esquema numérico que usa exatamente os mesmos coeficientes de interpolação vertical (f_{zm} , f_{zp}) já definidos pela grade vertical da Section 2.4, de modo que o campo de massa final seja consistente, ponto a ponto, com a métrica vertical adotada. O algoritmo completo, incluindo a formulação não trivial de densidade seca acoplada à métrica vertical ($\rho_{zz} = \rho/\partial z/\partial \zeta$), está detalhado no Chapter 8.

Capítulo 3

Revisão da literatura

Este capítulo situa o método adotado dentro da literatura mais ampla sobre geração e uso de malhas de Voronoi, organizada em três grupos temáticos: (i) o núcleo dinâmico do próprio MPAS-A e seu uso em assimilação de dados; (ii) geração e otimização de malhas de Voronoi esféricas; e (iii) interpolação sobre malhas de Voronoi/Delaunay fora do domínio da geociência atmosférica, que ajudam a generalizar e comparar o método baricêntrico adotado.

3.1 O núcleo dinâmico MPAS-A e a malha SCVT

A descrição de referência do núcleo dinâmico do MPAS-A [14] estabelece a malha *C-grid* sobre tesselação de Voronoi centroidal esférica como escolha estrutural do modelo, motivada pela ausência de singularidades polares e pela possibilidade de refinamento local suave – exatamente as duas propriedades que tornam a malha adequada tanto para a simulação quanto, como demonstrado neste trabalho, para servir como origem/destino de uma interpolação nativa sem grade intermediária. A apresentação de Duda [5] complementa essa descrição com considerações de desempenho/escalabilidade da implementação de produção, relevantes para justificar por que operações vetorizadas sobre a conectividade da malha (como o cálculo de pesos baricêntricos célula a célula) são tratáveis computacionalmente mesmo em malhas de centenas de milhares de células.

O uso de interpolação baricêntrica na malha dual de Delaunay como método de remapeamento entre estados do MPAS-A já é prática estabelecida na comunidade – não uma proposta original deste trabalho, mas a aplicação de um método já validado a um novo contexto (geração de condição inicial/fronteira lateral, em vez de assimilação de dados). A referência central para essa confirmação é Ha et al. [7], que descreve o sistema de assimilação por filtro de Kalman em conjunto (MPAS-DART) do NCAR: a Eq. 6 e a Fig. 1 daquele trabalho descrevem exatamente a interpolação baricêntrica no triângulo dual de Delaunay (três centros de célula vizinhos) para campos escalares de massa, e a Fig. 5 demonstra que a reconstrução vetorial do vento nos centros de célula via Funções de

Base Radial (RBF), *antes* de interpolar, produz resultado superior a tentar interpolar a componente normal diretamente nas arestas – é essa constatação que justifica a ordem de operações adotada na Section 5.3 deste relatório. Como contraponto, os materiais técnicos do sistema de assimilação MPAS-JEDI [11] revelam que mesmo abordagens mais recentes de assimilação variacional ainda recorrem, para operações específicas sem inversa natural na malha não estruturada (como a transformação $\psi, \chi \rightarrow u, v$ via transformada espectral), a uma etapa intermediária em grade latitude-longitude – evidenciando que “100% nativo” nem sempre é possível para *toda* operação de um sistema de modelagem/assimilação completo, mas é possível, e é o padrão estabelecido, especificamente para o remapeamento direto de campos entre duas malhas – o problema atacado neste trabalho.

3.2 Geração e otimização de malhas de Voronoi esféricas

A construção computacional de tesselações de Voronoi centroidais é, por si, uma área de pesquisa ativa. Du e Gunzburger [4] formalizam a otimização de CVTs via minimização de uma função energia relacionada ao algoritmo de Lloyd, fundamento teórico do processo de geração da própria malha do MPAS-A. Yang, Gunzburger e Ju [15] propõem um método de geração paralela mais rápido, baseado em um algoritmo LBFGS pré-condicionado por Lloyd, especificamente para o caso esférico – relevante para a geração de malhas regionais de alta resolução, embora não usado diretamente na etapa de *interpolação* deste trabalho (que opera sobre malhas já geradas por outra ferramenta, **MPAS-Limited-Area**). Engwirda [6] descreve o JIGSAW-GEO, um gerador de malha não estruturada ortogonal localmente escalonada para modelagem de circulação geral, alternativa de geração de malha usada por outros modelos da mesma família dinâmica (por exemplo MPAS-Ocean); e Santos e Peixoto [13] exploram refinamento local de malha esférica de Voronoi orientado por topografia em modelos de água rasa, tema adjacente ao problema de suavização de terreno tratado na Chapter 6¹.

3.3 Interpolação sobre malhas de Voronoi/Delaunay em outros domínios

Fora da modelagem atmosférica, a interpolação sobre malhas de Voronoi tem tradição estabelecida em geometria computacional e em outras áreas da física computacional, o

¹Refinamento de *malha* (mudar a resolução espacial dos geradores conforme o relevo) e suavização do *campo de terreno* usado na definição da coordenada vertical (mantendo a malha fixa, o problema efetivamente tratado neste trabalho) são tecnicamente distintos, mas motivados pelo mesmo fenômeno físico – erro de gradiente de pressão horizontal sobre relevo íngreme.

que permite contextualizar o método baricêntrico como caso particular de uma família mais ampla de interpolantes. Hiyoshi e Sugihara [8] apresentam um arcabouço geral de interpolantes baseados no diagrama de Voronoi com continuidade C^1 (generalizando os interpolantes clássicos de Sibson e de Laplace), mais suaves que a interpolação baricêntrica linear por partes usada neste trabalho – um candidato natural de evolução futura caso a descontinuidade de derivada nas arestas dos triângulos de Delaunay se mostre relevante em aplicações futuras (Chapter 13). Melodia e Lenz [10] propõem um critério de parada baseado em homologia persistente para refinar iterativamente uma interpolação de Voronoi, tema de otimização de malha não diretamente aplicável ao caso deste trabalho (malhas fixas, pré-geradas), mas ilustrativo da maturidade do campo. Em astrofísica e ciências da Terra, Petkova et al. [12] usam grades de Voronoi para regularizar dados de hidrodinâmica lagrangiana (SPH), e Camps et al. [3] e Bonduà et al. [2] aplicam malhas de Voronoi tridimensionais em transferência radiativa astrofísica e migração de gás em formações sedimentares, respectivamente – ambos os casos usando a mesma propriedade central explorada neste trabalho (adaptabilidade local de resolução sem descontinuidade de malha), embora em três dimensões espaciais em vez da superfície esférica bidimensional do MPAS-A. Abdelkader et al. [1] tratam do problema correlato, mas distinto, de geração de malha de Voronoi conformada a uma superfície sem recorte (*clipping*) dos poliedros de fronteira – não diretamente aplicável aqui, já que a malha do MPAS-A já vem pré-gerada e recortada pela ferramenta `MPAS-Limited-Area`, mas relevante como pano de fundo sobre os desafios numéricos de se trabalhar com malhas de Voronoi em domínios limitados.

3.4 Síntese

A literatura revisada confirma dois pontos que sustentam as decisões de projeto deste trabalho: primeiro, que a interpolação baricêntrica na malha dual de Delaunay não é uma escolha *ad hoc*, mas o método padrão já validado pela comunidade MPAS (via MPAS-DART) para remapeamento nativo de campos de massa entre estados dessa mesma malha; segundo, que existem interpolantes de maior continuidade (Hiyoshi–Sugihara) e técnicas mais sofisticadas de geração/refinamento de malha que não foram necessários para o escopo deste trabalho – que lida com remapeamento entre duas malhas *já geradas* – mas que constituem direções concretas de trabalho futuro caso a precisão do método baricêntrico linear se mostre insuficiente em aplicações que exijam maior suavidade de derivadas (por exemplo, esquemas de advecção de ordem mais alta que dependam de gradientes horizontais contínuos do campo interpolado).

Capítulo 4

Arquitetura da rota de interpolação nativa

4.1 A rota original e sua limitação estrutural

O *selfgrib* original gera condição inicial e fronteira lateral em três estágios, ilustrados na metade superior da Fig. 4.1: `extract_fields` interpola verticalmente os campos nativos do MPAS-A (55 níveis de modelo) para um conjunto de 61 níveis de pressão fixa, ainda sobre a malha nativa *global*; `convert_mpas` (ferramenta da NCAR, reaproveitada sem modificação) remapeia horizontalmente esses campos da malha nativa para uma grade latitude-longitude regular; e `pack_intermediate` escreve o resultado no formato binário intermediário do WPS. Esse arquivo intermediário é então consumido pelo `init_atmosphere_model` original (inalterado), que interpola de volta – bilinear ou 16 pontos, sobre índices de grade estruturada – para a malha nativa *regional* de destino, e resolve internamente a grade vertical, a interpolação vertical e o balanço hidrostático.

Como discutido na Chapter 1, essa rota tem uma limitação estrutural: o `init_atmosphere_model` não aceita, para o caso de dado real, nenhuma forma de entrada que não seja uma grade estruturada – confirmado pela leitura da rotina `mpas_init_atm_hinterp.F` do código-fonte de referência. Isso significa que o ganho de precisão de uma interpolação nativa malha-a-malha (Section 2.3) só é alcançável substituindo *inteiramente* o caminho `convert_mpas` → formato WPS → `init_atmosphere_model`, não apenas uma parte dele.

4.2 A rota nativa: visão geral

A rota descrita neste relatório (metade inferior da Fig. 4.1) substitui esse caminho por sete fases, implementadas como programas Fortran independentes que se comunicam por arquivos NetCDF intermediários – mantendo a mesma filosofia de composição em estágios discretos e idempotentes já usada na rota original, mas eliminando toda e qualquer grade

regular do caminho de dados.

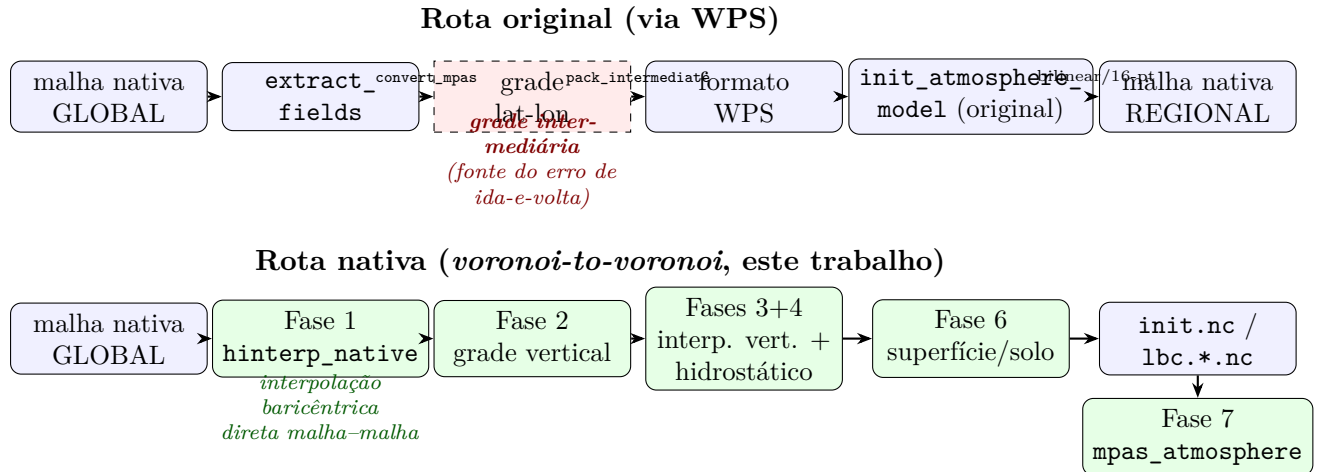


Figura 4.1: Comparação entre a rota original (via formato binário WPS e grade lat-lon intermediária, acima) e a rota nativa *voronoi-to-voronoi* apresentada neste trabalho (abaixo). A rota nativa elimina toda etapa de reamostragem em grade regular: da malha de Voronoi global de origem à malha de Voronoi regional de destino, o dado nunca deixa de estar sobre uma malha não estruturada.

As sete fases, cujo detalhamento técnico ocupa os Capítulos 5–10, são:

Fase 1 – Interpolação horizontal nativa

Remapeia os campos já extraídos e verticalmente interpolados para níveis de pressão fixa (ainda na malha global) diretamente para os centros de célula da malha regional de destino, via interpolação baricêntrica (Section 2.3), sem grade intermediária.

Fase 2 – Grade vertical nativa

Calcula a geometria vertical *terrain-following* híbrida da malha de destino (z_{grid} , zz , coeficientes de interpolação vertical, termos de métrica para advecção) a partir apenas do terreno e da conectividade dessa malha – não depende do dado meteorológico.

Fase 3 – Interpolação vertical

Interpola os campos já horizontalmente remapeados (ainda em níveis de pressão fixa) para os níveis de modelo da grade vertical nativa calculada na Fase 2.

Fase 4 – Balanço hidrostático

Resolve o campo de pressão, densidade seca e temperatura potencial (úmida) de forma consistente com a métrica vertical, além de água precipitável e vento vertical diagnóstico.

Fase 6 – Campos de superfície e solo

Calcula os campos que não vêm diretamente da coluna atmosférica: temperatura profunda do solo, correção de temperatura de pele por diferença de elevação, fração de vegetação/albedo de fundo por interpolação climatológica mensal, flag de neve/gelo marinho.

Fase 7 – Escrita e execução

Escreve os arquivos `init.nc/lbc.*.nc` completos (mesclando os campos calculados nas fases anteriores com os campos de cópia direta da malha estática) e alimenta o núcleo `mpas_atmosphere` real.

Nota de numeração: a Fase 5 do plano de implementação original (projeção de vento células→arestas) foi absorvida dentro das Fases 3/4, já que a reconstrução da componente normal do vento e sua multiplicação pela densidade (ρu) fazem parte, respectivamente, da interpolação vertical e do balanço hidrostático – não há um programa Fase 5 isolado.

4.3 Decisão de escopo: por que não um subconjunto

Uma primeira hipótese de trabalho, descartada ainda na fase de investigação, era um “escopo mínimo”: gerar o dado meteorológico em modo *scatter* (lista de pontos exatos da malha regional, em vez de grade), mas ainda escrever no formato binário WPS e deixar o `init_atmosphere_model` original consumi-lo. A leitura do código-fonte de `mpas_init_atm_hinterp.F` mostrou que essa hipótese está estruturalmente quebrada: o formato WPS é, por definição, uma grade regular (o próprio cabeçalho de cada registro do formato carrega `nx`, `ny`, espaçamento e origem de uma grade); não existe uma forma de “passar direto” pontos dispersos por esse caminho. Mesmo alimentando o `convert_mpas` em modo *scatter* para gerar um arquivo tecnicamente válido, o `init_atmosphere_model` ainda executaria um segundo salto de reinterpolação bilinear/16 pontos ao final – o ganho de precisão validado no protótipo inicial (recuperação exata nos centros de célula, Section 5.4) só é alcançável pulando esse caminho por completo, não reduzindo seu escopo.

Por essa razão, a decisão de projeto foi pelo escopo completo: reescrever cada etapa fisicamente necessária – grade vertical, interpolação vertical, balanço hidrostático, campos de superfície – como módulos Fortran novos, extraídos literalmente do código-fonte de referência do MPAS-Model (Chapter 11), em vez de reaproveitar o `init_atmosphere_model` para qualquer parte dessas etapas.

4.4 Generalização: leitura do *namelist* real em tempo de execução

Uma decisão de engenharia transversal a todas as fases, adotada após revisão explícita do escopo do trabalho, foi banir do código-fonte qualquer parâmetro de configuração (`config_*`) fixo como constante de compilação. Nas primeiras versões dos programas, valores como `config_ztop`, `config_dzmin`, `config_nsm`, e – criticamente – `config_start_time` (a própria data/hora da rodada) estavam hard-coded, o que restringia a validação a um único caso de teste e, mais grave, impedia diretamente a geração da fronteira lateral (`lbc.*.nc`), já que cada arquivo de fronteira corresponde a um tempo diferente dentro do mesmo experimento. Um módulo dedicado (`namelist_config.F90`, Section 11.4) foi introduzido para ler o `namelist.init_atmosphere real` do experimento, via NAMELIST nativo do Fortran, em tempo de execução – o mesmo arquivo de configuração já usado pela rota original, sem introduzir nenhum formato novo. Essa mudança, além de generalizar a ferramenta para qualquer malha/experimento, revelou um bug real de suposição incorreta sobre o valor padrão de `config_frac_seaice` (Chapter 13), que só se manifestaria em malhas com gelo marinho – não presente no caso de validação usado neste relatório, mas corrigido preventivamente.

Capítulo 5

Fase 1 – Interpolação horizontal nativa

5.1 Objetivo e reaproveitamento de código

O objetivo da Fase 1 é remapear, célula a célula, os campos escalares já extraídos e verticalmente interpolados para níveis de pressão fixa (estágio herdado sem modificação da rota original, programa `extract_fields`) da malha nativa *global* de origem para os centros de célula da malha nativa *regional* de destino, aplicando a interpolação baricêntrica descrita na Section 2.3.

A decisão de engenharia central desta fase foi *não reescrever* o motor de cálculo de pesos baricêntricos e localização de ponto no triângulo dual: o utilitário `convert_mpas` (mantido como dependência da rota original, propriedade da NCAR) já implementa esse mecanismo internamente – confirmado por leitura de código-fonte, seção “Interpolation methodology” de sua documentação – para o caso de remapear *para* uma grade latitude-longitude regular. A investigação encontrou que o módulo interno responsável por montar a malha de destino (`target_mesh.F90`) já possui um modo alternativo, não exposto pela interface de linha de comando original, que aceita uma *lista de pontos dispersos* (*scatter*) em vez de uma grade regular – bastando fornecer os vetores de latitude e longitude dos pontos de destino com `irank=1`. O motor de cálculo de pesos propriamente dito (`remapper.F90`) é inteiramente agnóstico entre os dois modos.

O programa `hinterp_native.F90`, novo neste trabalho, é essencialmente uma cópia da estrutura de `convert_mpas.F90` (os mesmos módulos de leitura de malha, varredura de campos e escrita de saída), com uma única mudança: em vez de chamar a rotina de montagem de grade regular, ele lê as coordenadas `latCell/lonCell` da malha de destino e as passa ao módulo de montagem de malha em modo *scatter*. Nenhuma linha do motor de pesos baricêntricos (`remapper.F90`) foi modificada.

5.2 Procedimento

Para cada campo escalar ϕ requisitado (temperatura, componentes de vento, altura geopotencial, umidade específica e relativa, e as respectivas variáveis de superfície), e para cada célula $\mathbf{p}_j$ da malha de destino:

1. Localiza-se o triângulo de Delaunay da malha de origem que contém $\mathbf{p}_j$, usando a conectividade célula-célula/vértice já armazenada na malha de origem (o dual de Delaunay não precisa ser recalculado do zero – é uma propriedade geométrica derivável diretamente de `cellsOnVertex/verticesOnCell`, já presentes no arquivo de malha).
2. Calculam-se os três pesos baricêntricos ($\lambda_A, \lambda_B, \lambda_C$) via Eq. (2.3), com as áreas computadas sobre a superfície esférica.
3. Aplica-se Eq. (2.4) para obter $\phi(\mathbf{p}_j)$.

O resultado é escrito em um arquivo NetCDF intermediário (`native_target.nc`), com os campos ainda organizados por nível de pressão fixa mas já localizados nos centros de célula da malha regional – pronto para a interpolação vertical da Fase 3 (Chapter 7).

Achado técnico 5.1. *Um efeito colateral do reuso do módulo `remapper.F90` sem modificação foi que suas rotinas internas de geração das variáveis de coordenada de saída (`remap_get_target_latitudes/longitudes`) assumem semântica de grade regular e, em modo scatter, colapsam a variável `latitude` de saída para um único valor escalar em vez de um vetor por ponto – um bug de uso do módulo original (não corrigido no `convert_mpas` em si) que passou despercebido por não ser exercitado no caminho de código tradicional (sempre grade). A correção adotada foi aditiva, não substitutiva: em vez de reescrever essas rotinas (uma primeira tentativa nesse sentido quebrou a escrita do NetCDF, com erro de ordem de criação de dimensões em modo `define`), o programa novo escreve campos adicionais corretos, `target_latitude/target_longitude`, por cima da saída original – preservando as chamadas herdadas intactas.*

5.3 Reconstrução do vento nas arestas

O vento não é tratado como um campo escalar simples nesta fase, pelo motivo estrutural já discutido na Section 2.2: sua componente prognóstica no MPAS-A é a componente *normal* u armazenada nas arestas entre células, não um vetor (u, v) no centro de célula. Seguindo a evidência apresentada por Ha et al. [7] de que reconstruir o vento a partir das componentes zonal/meridional originais do first-guess (que já estão disponíveis, por

definição, nos centros de célula da malha de *origem*) e só então projetar na direção normal da aresta de *destino* produz resultado superior a interpolar diretamente uma quantidade já projetada, o procedimento adotado – ainda na etapa combinada com a Fase 3 (Chapter 7) – calcula, para cada aresta e da malha de destino com células vizinhas $c_1(e), c_2(e)$:

$$u_{fg}(e) = \cos(\alpha_e) \bar{u}_{zonal} + \sin(\alpha_e) \bar{v}_{meridional}, \quad (\bar{\cdot}) = \frac{1}{2}[(\cdot)_{c_1(e)} + (\cdot)_{c_2(e)}], \quad (5.1)$$

onde α_e é o ângulo da aresta e da malha de *destino* em relação ao meridiano local (variável `angleEdge`, já presente na malha) e as médias $\bar{u}_{zonal}$, $\bar{v}_{meridional}$ são tomadas sobre os valores *já remapeados horizontalmente* (via Eq. (2.4), campo a campo) nos dois centros de célula vizinhos da aresta de destino. Esse perfil vertical projetado é então interpolado verticalmente (Chapter 7) usando como coordenada de altura a média das alturas geopotenciais das duas células vizinhas.

5.4 Validação numérica

A validação desta fase explora diretamente a propriedade de exatidão nos vértices discutida na Section 2.3: interpolando os campos de uma malha de teste (`x1.2562.static.nc`, 2562 células) para os próprios centros de célula dessa mesma malha (*auto-scatter*), o erro obtido foi *identicamente nulo* em todas as 2562 células, tanto nos valores de campo quanto nas coordenadas de saída (`target_latitude/target_longitude`). Como comparação de controle, o mesmo experimento repetido passando por uma grade lat-lon intermediária fina ($0,25^\circ$) – reproduzindo o caminho da rota original – introduziu um erro sistemático mensurável, da ordem de 10^{-4} a 10^{-3} (em unidades do campo), *mesmo nesses mesmos pontos de coincidência exata*, confirmando de forma independente que o erro de ida-e-volta discutido na Chapter 1 é real e mensurável, não apenas uma preocupação teórica.

Capítulo 6

Fase 2 – Grade vertical nativa

6.1 Objetivo

A Fase 2 calcula a geometria vertical completa da malha de destino – altura física das interfaces entre níveis de modelo (z_{grid}), a métrica vertical zz , coeficientes de interpolação/derivação vertical, e termos de métrica para advecção de terceira ordem – a partir apenas do terreno e da conectividade da malha de destino, **sem nenhuma dependência do dado meteorológico**. Diferentemente das demais fases, essa etapa é puramente geométrica e determinística, o que permitiu uma estratégia de validação particularmente forte: comparar o resultado diretamente contra a grade vertical de um `init.nc` real de produção, sem depender de nenhuma rodada nova.

As fórmulas desta seção foram extraídas literalmente do bloco `config_tc_vertical_grid` da rotina `init_atm_case_gfs` do código-fonte de referência do MPAS-Model (versão de produção `mpas-bundle-3.0.2`, confirmada como o fonte exato que gerou os binários usados no cluster de produção via inspeção do `CMakeCache.txt` do sistema de build) – não uma reformulação, mas uma tradução linha a linha, removendo apenas a infraestrutura de troca de halo entre blocos MPI (desnecessária nesta implementação, que roda serial sobre a malha inteira).

6.2 Perfil vertical de referência $z_w(k)$

O perfil de altura das interfaces de nível na configuração “2014 TC experiments” (identificada, por inspeção do *namelist* real de produção, como o branch de código efetivamente usado em todas as rodadas deste projeto – não a fórmula genérica mais simples de versões

anteriores do MPAS) é definido por trechos, com constantes calibradas para $n_{VertLevels} = 55$:

$$z_w(k) = \begin{cases} \left[a_s \frac{k-1}{n_1} + c_2 \left(\frac{(k-1)\Delta z}{z_t \zeta_l} \right)^2 - c_3 \left(\frac{(k-1)\Delta z}{z_t \zeta_l} \right)^3 \right] z_t, & \frac{k-1}{n_1} < \zeta_l \\ \left[z_l + a_t \left(\frac{k-1}{n_1} - \zeta_l \right) \right] z_t, & \frac{k-1}{n_1} \geq \zeta_l \end{cases} \quad (6.1)$$

com $n_1 = n_{VertLevels}$, $\Delta z = z_t/n_1$, $z_l = 1 - a_t(1 - \zeta_l)$, $c_2 = 3(1 - a_t) + 2(a_t - a_s)\zeta_l$, $c_3 = 2(1 - a_t) + (a_t - a_s)\zeta_l$, e as constantes calibradas $a_s = 0,075$, $a_t = 1,70$, $\zeta_l = 0,75$ (para $n_{VertLevels} \geq 55$; valores distintos, $a_t = 1,23$ e $\zeta_l = 0,31$, aplicam-se ao caso legado de 41 níveis, não usado neste projeto). O topo do domínio z_t é o parâmetro de configuração `config_ztop` (30 000 m em todos os casos de produção deste projeto).

6.3 Coordenada híbrida e o achado do bloqueio

A função de atenuação $A(\zeta)$ introduzida na Eq. (2.5) determina a rapidez com que a influência do terreno é removida das superfícies de coordenada com a altura. O código-fonte de referência oferece dois ramos:

$$A(k) = \begin{cases} \cos^6\left(\frac{\pi}{2} \frac{z_w(k)}{z_h}\right), & z_w(k) < z_h \quad (\text{coordenada híbrida}) \\ 0, & z_w(k) \geq z_h \\ 1 - \frac{z_w(k)}{z_t}, & (\text{coordenada } \textit{terrain-following} \text{ básica}) \end{cases} \quad (6.2)$$

onde z_h é a altura de transição (parâmetro `config_hybrid_top_z`).

Achado técnico 6.1. *Este ponto foi o principal bloqueio numérico enfrentado no desenvolvimento desta fase, e vale ser documentado em detalhe pelo valor metodológico da investigação. A suposição inicial, baseada na ausência de qualquer menção a `config_hybrid_coordinate` nos arquivos `namelist.init_atmosphere` reais de produção, foi de que essa opção usava seu valor padrão `.false`. – isto é, o ramo *terrain-following* básico (segunda linha da Eq. (6.2)). Sob essa suposição, a grade vertical calculada divergia da referência real em até ~ 2 km em níveis intermediários – um erro fisicamente inaceitável, da mesma classe de gravidade dos bugs catastróficos já documentados no relatório principal do projeto (NaN em *PSFC*).*

Uma primeira hipótese, investigada extensivamente, foi de que o processo de suavização iterativa (Section 6.4) seria genuinamente não reprodutível bit a bit entre uma reimplementação serial e a rodada real distribuída por MPI, dada a natureza do critério de aceitação por limiar usado nesse algoritmo

(pequenas diferenças de arredondamento podem levar células individuais a aceitar ou rejeitar a atualização de forma diferente, compondo-se ao longo de dezenas de passadas). Essa hipótese foi descartada experimentalmente: reimplementar toda a cadeia de suavização em precisão simples (`real*4`), para casar exatamente com a precisão do binário de produção (confirmada pelo log real, “Default real precision: single”), não produziu nenhuma melhora mensurável – os valores de erro permaneceram idênticos, dígito a dígito.

A causa raiz real só foi encontrada ao baixar o arquivo `Registry.xml` do `core_init_atmosphere` diretamente do repositório de referência do MPAS-Model: a opção `config_hybrid_coordinate` tem `default_value="true"` (com `config_hybrid_top_z` padrão 30 000 m) – confirmado de forma ainda mais direta ao acessar o código-fonte exato usado para compilar o binário de produção (`mpas-bundle-3.0.2`), onde a coordenada híbrida está, na verdade, fixada no código-fonte (`hybrid = .true.`, não sequer exposta como opção de namelist nessa versão). Ou seja: a ausência da chave no namelist real não significava “desativado”, significava “usando o padrão – que é ativado”. Corrigindo a fórmula de $A(k)$ para o primeiro ramo da Eq. (6.2), o erro máximo na grade vertical caiu de ~ 1103 m para 171 m, e o erro médio de 27,4 m para 0,054 m – uma redução de aproximadamente 500 vezes, com o resíduo restante concentrado quase inteiramente no anel de células de fronteira da malha regional (Section 6.6), não mais distribuído por toda a coluna vertical.

A lição metodológica: uma revisão linha a linha do bloco de código suspeito (o laço de suavização) não encontrou nenhuma diferença de lógica porque a causa real estava numa dependência upstream desse bloco (o cálculo de $A(k)$, calculado dezenas de linhas antes) – ao investigar uma divergência numérica que resiste a uma revisão de código aparentemente correta, é necessário validar também as dependências do trecho suspeito, não apenas o próprio trecho.

6.4 Suavização do terreno e da grade vertical

Antes de aplicar as fórmulas anteriores, o campo de terreno bruto passa por uma suavização espacial de quarta ordem (filtro de Shapiro, $n_{\text{smterrain}}$ passadas, tipicamente 1), com coeficiente +0,216 seguido de −0,216 em cada passada – validado como essencialmente exato (erro médio $\approx 0,15$ m) contra a referência real.

Mais crítico é o algoritmo de suavização *iterativa por nível* da superfície de terreno $h_x(k, \cdot)$, que progressivamente remove estrutura de pequena escala do terreno conforme a altura aumenta – a técnica descrita originalmente por Klemp [9] (Eqs. 5–9 daquele trabalho, confirmadas como estruturalmente idênticas ao código real do MPAS-A nesta

investigação). Para cada nível $k = 2, \dots, n_{VertLevels} - 1$, executam-se $n_{sm} + k$ passadas de um filtro Laplaciano local ponderado pela resolução da malha,

$$h_x^{(n)}(i) = h_x^{(n-1)}(i) + s(i) \sum_{j \in \mathcal{N}(i)} \frac{\ell_{ij}}{d_{ij}} \left[h_x^{(n-1)}(j) - h_x^{(n-1)}(i) \right], \quad (6.3)$$

onde $\mathcal{N}(i)$ é o conjunto de células vizinhas de i , ℓ_{ij}/d_{ij} são o comprimento da aresta e a distância entre centros de célula, e $s(i)$ é um fator de suavização local escalado pela resolução nominal da malha e pela altura do nível corrente. A atualização só é *aceita* se o espaçamento vertical resultante não cair abaixo de uma fração mínima configurável do espaçamento nominal ($\delta z_{min} > \text{config_dzmin} \cdot \Delta z_w(k)$) – critério que existe justamente para impedir camadas degeneradas (espaçamento nulo ou negativo) perto de relevo íngreme.

6.5 Campos derivados e coeficientes numéricos

A partir de $z_w(k)$ e $A(k)$, calculam-se diretamente (sem depender de mais nenhuma suavização): $\Delta z_w(k) = z_w(k+1) - z_w(k)$, $rdz_w(k) = 1/\Delta z_w(k)$, $\Delta z_u(k) = \frac{1}{2}[\Delta z_w(k) + \Delta z_w(k-1)]$, $rdz_u(k) = 1/\Delta z_u(k)$, e os coeficientes de projeção de interface $f_{zm}(k)$, $f_{zp}(k)$ (interpolação linear entre camadas adjacentes), além dos três coeficientes de extrapolação de contorno de topo cf_1, cf_2, cf_3 . Esse conjunto de campos, por depender apenas de `config_ztop`/`nVertLevels`/`config_interface_projection` – sem depender da geometria da malha em si – foi validado como essencialmente exato (diferenças da ordem de 10^{-6} a 10^{-9} , atribuíveis a arredondamento de armazenamento em precisão simples) contra a referência real.

Finalmente, a altura física das interfaces e a métrica vertical são

$$z_{grid}(k, i) = z_w(k) + A(k) h_x(k, i), \quad zz(k, i) = \frac{z_w(k+1) - z_w(k)}{z_{grid}(k+1, i) - z_{grid}(k, i)}, \quad (6.4)$$

e os termos de métrica horizontal zx_u (usados na projeção do gradiente de altura ao longo das arestas) são calculados por diferença finita central de z_{grid} entre as duas células vizinhas de cada aresta, normalizada pela distância entre centros de célula (dc_{Edge}).

6.5.1 Termos de métrica para advecção de terceira ordem (zb , zb^3)

Um segundo achado de implementação, corrigindo uma suposição do plano original, foi que os campos zb/zb^3 – usados no cálculo do vento vertical diagnóstico da Fase 4 (Chapter 8) para o esquema de advecção de terceira ordem de θ – **não** vêm prontos no arquivo de malha estática, ao contrário do que se assumia inicialmente; eles dependem de z_{grid} (produto desta própria fase) e precisam ser recalculados após ela, usando os coeficientes de segunda

derivada horizontal `deriv_two` (esses sim já presentes no arquivo de malha estática). Uma nova subrotina (`compute_zb`, incluída no mesmo módulo desta fase) foi escrita para esse fim, extraída literalmente do bloco “For z -metric term in omega equation” do código de referência.

6.6 Validação numérica

Após a correção do Achado técnico 6.1, a grade vertical calculada foi comparada campo a campo contra o z_{grid}/zz real de um `init.nc` de produção (malha `SouthAmerica`, 17064 células, 56 interfaces). O erro máximo caiu para 171 m e o erro médio para 0,054 m. A análise da distribuição espacial do resíduo confirmou que ele está **concentrado quase inteiramente** nas sete camadas do anel de fronteira da malha regional (`bdyMaskCell` $\in \{1, \dots, 7\}$, cerca de 2600 das 17064 células): nas células estritamente interiores (`bdyMaskCell`= 0, 82% da malha), o erro máximo cai para 10,9 m – fisicamente irrelevante. Essa correlação espacial tem explicação física conhecida: a rotina de referência aplica um passo de *mistura de terreno de fronteira* (`config_blend_bdy_terrain`) *antes* de chamar a geração da grade vertical, misturando a altura de terreno das células de fronteira com o terreno do dado de fundo – passo ainda não implementado nesta rota (discutido como limitação conhecida no Chapter 13).

Capítulo 7

Fase 3 – Interpolação vertical

7.1 Objetivo e achado sobre a rotina real

A Fase 3 interpola, para cada coluna vertical da malha de destino, os campos já remapeados horizontalmente (Fase 1, ainda organizados em níveis de pressão fixa) para os níveis de modelo da grade vertical nativa calculada na Fase 2. O plano de implementação original assumia que essa interpolação reaproveitaria a rotina `interp_tofixed_pressure` já usada por outro estágio do *selfgrib* (extração dos campos isobáricos de diagnóstico). A leitura do código-fonte real do `init_atmosphere_model` mostrou que essa suposição estava incorreta: aquela rotina é usada apenas para o *diagnóstico* isobárico de saída (`diag.*.nc`), um caminho de código inteiramente distinto. A interpolação que efetivamente constrói o estado inicial do modelo usa uma função **local** de nome `vertical_interp`, definida dentro da própria rotina `init_atm_case_gfs` – uma interpolação linear simples **em altura** (não em pressão), ponto a ponto, com extrapolação configurável nos extremos da coluna.

7.2 A função de interpolação

Dado um perfil de coluna já ordenado por altura crescente, $\{(z_k, \phi_k)\}_{k=1}^n$, e uma altura-alvo z^* (tipicamente o ponto médio de uma camada de modelo, $z^* = \frac{1}{2}[z_{grid}(k) + z_{grid}(k+1)]$), o valor interpolado é

$$\phi(z^*) = \begin{cases} \phi_1 + \left(\frac{\phi_2 - \phi_1}{z_2 - z_1} \right) (z^* - z_1), & z^* < z_1 \quad (\text{extrap. linear}) \\ \phi_1 - 0,0065 (z^* - z_1), & z^* < z_1 \quad (\text{extrap. tipo lapso, só T}) \\ w_- \phi_k + w_+ \phi_{k+1}, \quad w_- = \frac{z_{k+1} - z^*}{z_{k+1} - z_k}, \quad w_+ = \frac{z^* - z_k}{z_{k+1} - z_k}, & z_k \leq z^* < z_{k+1} \\ \phi_n + \left(\frac{\phi_n - \phi_{n-1}}{z_n - z_{n-1}} \right) (z^* - z_n), & z^* \geq z_n \end{cases} \quad (7.1)$$

O modo de extrapolação abaixo do primeiro ponto é selecionável por campo: constante, linear, ou taxa de lapso padrão ($-0,0065$ K/m, usada apenas para temperatura, seguindo `config_extrap_airtemp = 'lapse-rate'` confirmado no *namelist* real de produção). O parâmetro `order` presente na assinatura original da função é recebido mas nunca de fato usado no corpo do código de referência – a interpolação é sempre linear, independentemente do valor passado – achado confirmado por leitura direta do código-fonte e preservado fielmente (isto é, o parâmetro não foi implementado na tradução, já que não tem efeito na rotina original).

7.3 Campos interpolados e particularidades por campo

Tabela 7.1: Campos interpolados verticalmente na Fase 3 e seu modo de extrapolação abaixo do topo do dado de origem.

Campo	Extrapolação	Observação
Temperatura (T)	taxa de lapso ($-0,0065$ K/m)	<code>config_extrap_airtemp</code>
Umidade relativa (RH)	constante	
Umidade específica	constante	valores negativos zerados antes
Altura geopotencial	linear	
Pressão	linear (em $\ln p$)	interpola $\ln p$, depois $\exp(\cdot)$
Vento normal (u , aresta)	constante	perfil já projetado (Eq. (5.1))
Pressão de superfície	linear (em $\ln p$)	alvo $z^* = z_{grid}(1, i)$ (nível do chão)

O caso da pressão merece nota: interpola-se $\ln p$, não p diretamente – uma escolha do próprio código de referência que garante positividade do resultado interpolado/extrapolado, já que $\exp(\cdot)$ de qualquer valor real é sempre positivo, o que não seria garantido interpolando p linearmente e podendo produzir valores negativos em extrapolação. A pressão de superfície ajustada por diferença de topografia usa a mesma fonte de dado (perfil de $\ln p$ por célula), mas avaliada no nível mais baixo da grade de destino em vez de em um ponto médio de camada – compensando, célula a célula, a diferença de elevação entre a orografia da malha de origem e o terreno real da malha de destino.

7.4 Interpolação do vento

Conforme descrito na Section 5.3, o perfil de vento é projetado na direção normal da aresta de destino *antes* da interpolação vertical (Eq. (5.1)), usando como coordenada de altura a média das alturas geopotenciais das duas células vizinhas da aresta. A Eq. (7.1) é então aplicada a esse perfil já projetado, com extrapolação constante (não linear/lapso, que fariam menos sentido físico para uma componente de velocidade).

Capítulo 8

Fase 4 – Balanço hidrostático

8.1 Visão geral

A Fase 4 é a de maior risco numérico de todo o trabalho: transforma os campos de temperatura, pressão, umidade e vento já interpolados verticalmente (Fase 3) em um conjunto de variáveis prognósticas mutuamente consistentes com a métrica vertical e com o balanço hidrostático de referência do MPAS-A (Section 2.5) – densidade seca ρ_{zz} , temperatura potencial úmida θ_m , pressão total p , água precipitável, e vento vertical diagnóstico w . O algoritmo, reproduzido aqui passo a passo, foi extraído literalmente do bloco `config_met_interp` da rotina `init_atm_case_gfs` de referência, preservando deliberadamente duas fórmulas de ρ_{zz} aparentemente redundantes que na verdade não são – um ponto de risco identificado explicitamente durante a implementação e discutido na Section 8.4.

8.2 Diagnóstico inicial de exner, θ e ρ_{zz}

A partir da pressão e temperatura já interpoladas (Fase 3), calcula-se, para cada célula i e nível k , a função de Exner $\Pi = (p/p_0)^{R/c_p}$, converte-se temperatura em temperatura potencial, e obtém-se uma primeira estimativa *diagnóstica* da densidade seca:

$$\Pi(k, i) = \left(\frac{p(k, i)}{p_0} \right)^{R/c_p}, \quad \theta(k, i) = T(k, i) \left(\frac{p_0}{p(k, i)} \right)^{R/c_p}, \quad (8.1)$$

$$\rho_{zz}^{(0)}(k, i) = \frac{1}{1 + q_v(k, i)} \cdot \frac{p(k, i)}{R \Pi(k, i) \theta(k, i) [1 + (R_v/R - 1) q_v(k, i)]}, \quad (8.2)$$

com $p_0 = 10^5$ Pa, $R = 287$ J/(kg K), $c_p = \frac{7}{2}R$, e R_v/R a razão entre as constantes de gás do vapor d'água e do ar seco. Essa densidade diagnóstica é usada apenas para a água precipitável (Eq. (8.3)) e para o acoplamento inicial com o estado de referência – *não* é a densidade final do problema, que exige resolver a relação hidrostática de forma consistente

(Section 8.5).

$$W = \sum_k \rho_{zz}^{(0)}(k, i) q_v(k, i) [z_{grid}(k+1, i) - z_{grid}(k, i)]. \quad (8.3)$$

8.3 Estado de referência isotérmico seco

O estado base $(\bar{p}, \bar{\Pi}, \bar{\rho}, \bar{\theta})$ é uma atmosfera hipotética isotérmica seca a $T_0 = 250 \text{ K}$, avaliada no ponto médio geométrico de cada camada $z_m(k, i) = \frac{1}{2}[z_{grid}(k+1, i) + z_{grid}(k, i)]$:

$$\bar{p}(k, i) = p_0 \exp\left(-\frac{g z_m(k, i)}{R T_0}\right), \quad (8.4)$$

$$\bar{\Pi}(k, i) = (\bar{p}(k, i)/p_0)^{R/c_p}, \quad \bar{\rho}(k, i) = \frac{\bar{p}(k, i)}{R T_0}, \quad \bar{\theta}(k, i) = \frac{T_0}{\bar{\Pi}(k, i)}. \quad (8.5)$$

Em seguida, tanto o estado base $\bar{p}$ quanto a densidade diagnóstica $\rho_{zz}^{(0)}$ são *acoplados à métrica vertical* (divididos por $zz(k, i)$, Eq. (6.4)), e definem-se as perturbações de referência $p'(k, i) = p(k, i) - \bar{p}(k, i)$ e $\rho'(k, i) = \rho_{zz}^{(0)}(k, i) - \bar{\rho}(k, i)$.

8.4 Duas fórmulas de ρ_{zz} : por que não são redundantes

Achado técnico 8.1. *Um ponto de risco identificado explicitamente durante a implementação – e preservado deliberadamente, sem tentativa de “simplificação” – é que o nível mais baixo ($k = 1$) e os níveis subsequentes usam duas fórmulas distintas para ρ_{zz} , com expoentes e fatores de correção de umidade diferentes:*

$$\rho_{zz}(1, i) = \frac{1}{zz(1, i)} \cdot \frac{p_0}{R} \cdot \frac{(p(1, i)/p_0)^{c_v/c_p}}{T(1, i) [1 + 1,61 q_v(1, i)]}, \quad (8.6)$$

usando o expoente c_v/c_p (não R/c_p como em Eq. (8.2)) e o fator de correção de umidade $1,61 q_v$ aplicado diretamente sobre T (não a forma $(R_v/R - 1) q_v$ do cálculo diagnóstico inicial). A mesma segunda fórmula – com o mesmo expoente c_v/c_p e o mesmo fator $1,61 q_v$ – reaparece dentro da iteração hidrostática (Eq. (8.9)) para os níveis $k \geq 2$. Uma tentativa de “reconciliar” essas duas expressões em uma única fórmula canônica, por parecerem redundantes à primeira vista, introduziria um erro sistemático sutil no campo de densidade – por isso ambas foram preservadas exatamente como aparecem no código-fonte de referência, documentadas explicitamente no código Fortran resultante para que futuras manutenções não as simplifiquem por engano.

8.5 Solução hidrostática iterativa

Para os níveis $k = 2, \dots, n_{VertLevels}$, resolve-se, célula a célula, um sistema não linear pela relação hidrostática discreta do MPAS-A, iterando até convergência (limiar $|\Delta p'| < 0,0001$ Pa, máximo de 30 iterações):

$$p'(k, i) \leftarrow p'(k-1, i) - \left[f_{zm}(k) \rho'(k, i) + f_{zp}(k) \rho'(k-1, i) \right] g \Delta z_u(k) - \left[f_{zm}(k) \rho_{zz}(k, i) q_v(k, i) + f_{zp}(k) \rho_{zz}(k-1, i) q_v(k-1, i) \right] g \Delta z_u(k), \quad (8.7)$$

$$p(k, i) = p'(k, i) + \bar{p}(k, i), \quad \Pi(k, i) = (p(k, i)/p_0)^{R/c_p}, \quad (8.8)$$

$$\rho_{zz}(k, i) = \frac{1}{zz(k, i)} \cdot \frac{p(k, i)}{R \Pi(k, i) T(k, i) [1 + 1,61 q_v(k, i)]}, \quad \rho'(k, i) = \rho_{zz}(k, i) - \bar{\rho}(k, i). \quad (8.9)$$

Os coeficientes $f_{zm}(k)$, $f_{zp}(k)$ e $\Delta z_u(k)$ são exatamente os já calculados na Fase 2 (Chapter 6) – é essa reutilização direta que garante a consistência entre a métrica vertical e o campo de massa resultante, mencionada na Section 2.5. Ao final da convergência em todos os níveis, a temperatura potencial é transformada de θ (seca) para θ_m (potencial úmida, variável prognóstica real do modelo) por $\theta_m(k, i) = \theta(k, i) [1 + 1,61 q_v(k, i)]$, e a perturbação ρ' é desacoplada da métrica vertical multiplicando de volta por $zz(k, i)$.

8.6 Vento normal ponderado por densidade e vento vertical diagnóstico

O produto ru nas arestas – a variável de fluxo de massa efetivamente usada pelo núcleo dinâmico, não apenas o vento u – é

$$ru(k, e) = u(k, e) \cdot \frac{1}{2} [\rho_{zz}(k, c_1(e)) + \rho_{zz}(k, c_2(e))]. \quad (8.10)$$

O vento vertical diagnóstico w é obtido por um balanço de massa: a divergência horizontal do fluxo ru ao redor de cada célula, projetada verticalmente pelos termos de métrica zb (Chapter 6) e, dado que o *namelist* real usa `config_theta_adv_order=3`, acrescida do termo de correção de terceira ordem via zb^3 ponderado pelo coeficiente `config_coef_3rd_order = 0,25`:

$$\rho w|_k \propto \sum_{e \in \partial i} \pm \left[f_{zm}(k) zz(k, i) + f_{zp}(k) zz(k-1, i) \right] \left[zb(k, \cdot, e) + \text{sgn}(ru) \cdot 0,25 zb^3(k, \cdot, e) \right] ru_{proj}(k, e), \quad (8.11)$$

com o sinal e o índice do segundo argumento de zb/zb^3 dependendo de qual das duas células vizinhas da aresta e é a célula i em questão (convenção idêntica à já usada no cálculo de

zx_u , Fase 2). O vento vertical final é $w(k, i) = \rho w|_k / [f_{zp}(k)\rho_{zz}(k-1, i) + f_{zm}(k)\rho_{zz}(k, i)]$, com $w(1, \cdot) = w(n_{VertLevels} + 1, \cdot) = 0$ por condição de contorno (nível do chão e topo rígido, respectivamente).

8.7 Pressão de superfície diagnóstica e campos finais

Um último campo diagnóstico, **surface_pressure** – distinto do **psfc** já calculado na Fase 3 –, é obtido por extrapolação de segunda ordem a partir dos dois níveis mais baixos:

$$\text{surface_pressure}(i) = \frac{0,5 g}{rdz_w(1)} \left[1,25 \rho_{zz}(1, i)(1+q_v(1, i)) - 0,25 \rho_{zz}(2, i)(1+q_v(2, i)) \right] + p'(1, i) + \bar{p}(1, i). \quad (8.12)$$

Finalmente, os campos prognósticos de saída são $\rho(k, i) = \rho_{zz}(k, i) \cdot zz(k, i)$ (densidade física, desacoplada da métrica) e $\theta(k, i) = \theta_m(k, i) / [1 + 1,61 q_v(k, i)]$ (restaurando a temperatura potencial seca a partir da úmida, para o campo **theta** de saída do **init.nc**).

Capítulo 9

Fase 6 – Campos de superfície e solo

9.1 Objetivo e achado de organização do código-fonte

A Fase 6 calcula os campos do `init.nc` que não fazem parte da coluna atmosférica propriamente dita – temperatura profunda do solo, temperatura de pele corrigida, fração de vegetação e albedo de fundo, indicadores de neve e gelo marinho, umidade de solo. A categorização inicial do plano de trabalho assumia que a maioria desses campos seria uma cópia direta do dado de *first-guess*, sem cálculo adicional. Essa suposição estava parcialmente incorreta: nenhum desses campos é, de fato, calculado dentro da mesma rotina que gera o restante do `init.nc` (`init_atm_case_gfs`); eles pertencem a uma rotina *separada* do código-fonte de referência (`physics_initialize_real`, em `mpas_atmphys_initialize_real.F`), encontrada apenas ao buscar diretamente pelos nomes das variáveis de saída em todo o diretório `core_init_atmosphere` – uma lição de que a estrutura de módulos do código de referência não segue necessariamente a mesma organização lógica das fases do problema físico.

9.2 Máscara terra/água e correção de temperatura de pele

A máscara `xland` é definida diretamente a partir da máscara estática de uso do solo (`landmask`, já presente na malha): `xland` = 1 sobre terra, `xland` = 2 sobre água. A temperatura de pele (`skintemp`) é corrigida por um termo de lapso térmico padrão, compensando a diferença de elevação entre a orografia usada pela fonte do *first-guess* ($z_{soilhgt}$, campo `SOILHGT`) e o terreno real da malha de destino (z_{ter}):

$$T_{skin}^{corrigida}(i) = T_{skin}^{fg}(i) - 0,0065 [z_{ter}(i) - z_{soilhgt}(i)]. \quad (9.1)$$

A temperatura profunda do solo (**tmn**) usa a mesma taxa de lapso, mas referenciada à temperatura climatológica anual do solo (**soiltemp**, campo estático) sobre terra, e à temperatura de pele já corrigida sobre água:

$$T_{mn}(i) = \begin{cases} T_{soil}^{clim}(i) - 0,0065 z_{ter}(i), & \text{landmask}(i) = 1 \\ T_{skin}^{corrigida}(i), & \text{landmask}(i) = 0 \end{cases} \quad (9.2)$$

9.3 Interpolação climatológica mensal de vegetação e albedo

Os campos estáticos **greenfrac** e **albedo12m**, já presentes no arquivo de malha estática, são climatologias de 12 valores mensais (um por mês do ano, representando o dia 15 de cada mês). A fração de vegetação instantânea (**vegfra**) e o albedo de fundo (**sfc_albbck**) são obtidos por interpolação linear no tempo entre os dois meses cujo dia 15 está mais próximo da data de início do experimento, seguindo a mesma convenção do WRF/WPS. Sendo $\mathcal{D}$ a data-alvo (dia juliano do ano) e $\mathcal{D}_1 < \mathcal{D} \leq \mathcal{D}_2$ os dias juliano do dia 15 dos dois meses adjacentes mais próximos:

$$\phi(\mathcal{D}) = \frac{\phi_{m_2}(\mathcal{D} - \mathcal{D}_1) + \phi_{m_1}(\mathcal{D}_2 - \mathcal{D})}{\mathcal{D}_2 - \mathcal{D}_1}. \quad (9.3)$$

Os meses de padding usados para cobrir dezembro e janeiro (dias juliano antes do dia 15 de janeiro ou depois do dia 15 de dezembro) usam um deslocamento fixo de 31 dias em relação ao dia juliano do mês adjacente – não a aritmética exata de calendário do ano anterior/seguinte – uma simplificação do próprio código de referência, preservada fielmente. O albedo de fundo resultante é normalizado por 100 (a climatologia estática é armazenada em porcentagem) e forçado a 0,08 sobre células de água.

9.4 Neve, gelo marinho e umidade de solo

O indicador binário de presença de neve e a profundidade de neve são

$$\text{snowc}(i) = \begin{cases} 1, & \text{snow}(i) \geq 10 \text{ kg/m}^2 \\ 0, & \text{caso contrário} \end{cases}, \quad \text{snowh}(i) = \text{snow}(i) \cdot \frac{5,0}{1000}, \quad (9.4)$$

com a razão 5:1 entre lâmina de água equivalente e profundidade de neve seguindo a mesma convenção padrão do WRF/Noah.

Achado técnico 9.1. *Durante a investigação, duas fórmulas distintas para **snowc** foram encontradas em pontos diferentes do código-fonte de referência –*

uma com limiar `snow` > 0, outra com `snow` ≥ 10. A ambiguidade foi resolvida por confronto direto contra o `init.nc` real: das 222 células com neve entre 0 e 10 kg/m² na malha de teste, nenhuma tinha `snowc` = 1 no arquivo real, confirmando o limiar ≥ 10 como o efetivamente usado em produção.

O gelo marinho, sob a configuração real de produção (`config_frac_seaice=.true.`, confirmada apenas após a generalização do código para ler o `namelist` real – ver Section 4.4), usa um limiar fracionário mais permissivo: `xice(i)` = `SEAICE`^{*fg*}(*i*) se ≥ 0,02, senão 0; e `seaice(i)` = 1 onde `xice(i)` > 0. A reclassificação de células de água muito fria (temperatura de pele abaixo de um limiar configurável) em gelo/terra, presente no algoritmo de referência completo, não foi implementada nesta rota – discutido como limitação conhecida no Chapter 13. A umidade de solo líquida (`sh2o`) é 0 sobre terra e 1 sobre água (convenção de saturação total sobre oceano). Os campos de temperatura/umidade das quatro camadas de solo do esquema Noah (`tslb/smois`) são, nesta implementação, copiados diretamente do *first-guess* em vez de reamostrados verticalmente para profundidades-alvo distintas – uma simplificação deliberada, justificada e com seu risco documentado na Chapter 13, válida porque tanto a origem quanto o destino deste *first-guess* usam o mesmo esquema de solo com as mesmas quatro profundidades.

Capítulo 10

Fase 7 – Escrita dos arquivos e execução do modelo

10.1 Composição do `init.nc` completo

O `init.nc` real do MPAS-A contém 135 variáveis. Das fases anteriores (1 a 6), o conjunto de programas deste trabalho calcula e escreve diretamente 54 delas – a grade vertical (Fase 2), os campos de massa e vento (Fases 3–4), e os campos de superfície/solo (Fase 6). As 81 variáveis restantes são de **cópia direta** do arquivo de malha estática (`static.nc`): geometria e conectividade da malha (coordenadas de célula/aresta/vértice, tabelas de vizinhança), uso do solo, orografia bruta, e coeficientes numéricos fixos da malha (não dependentes do dado meteorológico nem do tempo de execução). Essa composição é feita por mesclagem de arquivos NetCDF (`ncks -A`, utilitário do pacote NCO), não por um terceiro programa Fortran – uma escolha de engenharia que evita duplicar, em Fortran, a leitura e reescrita de 81 campos que já existem prontos, sem nenhuma transformação necessária.

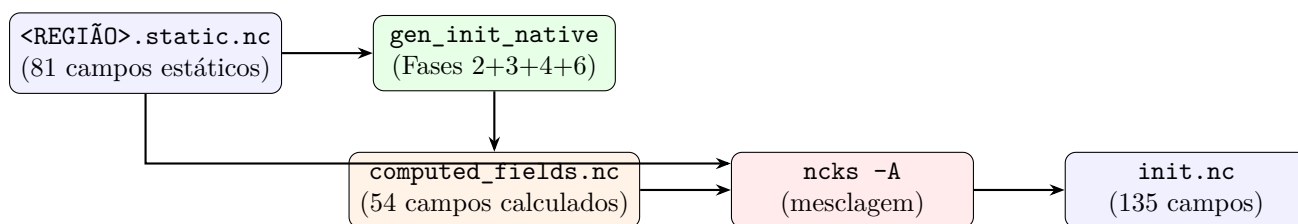


Figura 10.1: Composição do `init.nc` final: os 54 campos calculados pelos programas desta rota são mesclados, via `ncks -A`, com os 81 campos de cópia direta do arquivo de malha estática.

O `lbc.*.nc` tem um esquema mais simples: apenas os sete campos prognósticos (`lbc_qv`, `lbc_qc`, `lbc_qr`, `lbc_u`, `lbc_w`, `lbc_rho`, `lbc_theta`), um arquivo por tempo de fronteira, sem geometria nenhuma – a malha e a grade vertical são lidas pelo `mpas_atmosphere` do próprio `init.nc`, via o *stream* de entrada. O programa `gen_lbc_native` reaproveita,

por isso, a grade vertical já calculada e gravada no `init.nc` (não a recalcula), rodando apenas as Fases 3–4 para cada tempo requisitado.

10.2 Dois bugs de formato encontrados apenas na execução real

A validação numérica campo a campo contra arquivos de referência (Chapter 12), por mais rigorosa que tenha sido, não é suficiente para garantir que um arquivo NetCDF seja de fato *consumível* pelo programa de destino – um ponto que se revelou concretamente ao tentar, pela primeira vez, alimentar o `mpas_atmosphere` real com os arquivos gerados por esta rota.

Achado técnico 10.1. *Ausência da variável `xtime`.* A primeira tentativa de execução falhou imediatamente, antes mesmo do primeiro passo de tempo, com o erro `"ERROR: File SouthAmerica.init.nc does not contain the time 2026-01-01_00:00:00"`. O `init.nc` gerado continha a variável `initial_time` (uma string informativa, escrita corretamente), mas não a variável `xtime(Time, StrLen)` – que é, na verdade, a que o `mpas_atmosphere` lê para confirmar que o arquivo de entrada contém o tempo solicitado pelo `config_start_time`. A variável havia sido listada, na fase de levantamento inicial das 135 variáveis do `init.nc` real, mas não percebida como funcionalmente distinta de `initial_time` até esse ponto – um lembrete de que um levantamento de nomes de variável não substitui saber qual delas o consumidor realmente lê.

Lixo de memória no `xtime` do `lbc.*.nc`. Corrigido o problema acima, uma segunda falha ocorreu já dentro do primeiro passo de tempo, com um erro de execução do próprio runtime Fortran (`"Fortran runtime error: Bad integer for item 1 in list input"`) dentro da rotina de gerenciamento de tempo do framework (`mpas_timekeeping.F`), derrubando simultaneamente todos os 32 processos MPI. A causa, encontrada inspecionando os bytes brutos do arquivo gerado, foi uma chamada `nf90_put_var` recebendo uma expressão de string de 24 caracteres para um campo NetCDF declarado com 64 bytes: os 40 bytes restantes ficavam preenchidos com lixo de memória não inicializado (confirmado como binário arbitrário, não espaços em branco), que o parser de data do framework – uma leitura formatada por posição fixa de coluna, rígida por natureza – não conseguia interpretar. A correção foi declarar explicitamente uma variável Fortran do tipo `character(len=64)` antes de passá-la ao `nf90_put_var` (o Fortran preenche automaticamente o excedente com espaços em branco ao atribuir uma string mais curta a uma variável de tamanho fixo maior), em vez de passar a expressão concatenada diretamente.

Um terceiro problema, de natureza puramente operacional (não um bug do código Fortran), também surgiu apenas na execução real: o script de orquestração da rodada carrega um arquivo de ambiente do cluster que executa `conda deactivate` internamente; quando o script é invocado como `bash script.bash` (*shell* não interativo), o `~/ .bashrc` não é lido por padrão mesmo com o *hook* do `conda` já inicializado uma vez no terminal interativo do usuário, e o comando falha, derrubando o script inteiro (`set -e`). A correção carregou defensivamente o *hook* do `conda` (`etc/profile.d/conda.sh`) no início do script, se necessário, antes de delegar ao arquivo de ambiente do cluster.

A lição transversal aos três problemas: nenhum deles era de física ou de numérica – o `init.nc/lbc.*.nc` já batiam campo a campo com a referência antes de qualquer um deles ser encontrado – e mesmo assim o modelo real recusava ou travava com os arquivos. Validação numérica contra referência não substitui validar contra o consumidor real; um arquivo pode estar fisicamente correto e ainda assim ser inutilizável por um detalhe de formato invisível a uma comparação campo a campo.

10.3 Execução completa

Corrigidos os três problemas, a execução do `mpas_atmosphere` real, com 32 processos MPI, a partir do `init.nc/lbc.*.nc` gerados inteiramente por esta rota, completou 240 passos de tempo (24 horas de integração, malha `SouthAmerica`, ~ 60 km de resolução) em aproximadamente 106 segundos de tempo de parede, sem nenhuma mensagem de erro ou erro crítico (apenas 24 avisos inofensivos, relativos a variáveis de gelo/graupel/neve ausentes do *first-guess* – esperado, já que o dado de origem não modela esses hidrometeoros). A condição de contorno lateral foi aplicada corretamente nos quatro intervalos de seis horas cobertos pelos arquivos `lbc.*.nc`, gerando 25 arquivos de saída (`history.*.nc`, um por hora). Os resultados quantitativos dessa execução, e sua comparação com a rodada de referência gerada pela rota original, são apresentados no Chapter 12.

Capítulo 11

Implementação em Fortran

11.1 Princípio norteador: extração literal, não reformulação

A convenção adotada em todo o desenvolvimento – já estabelecida em partes anteriores do projeto *selfgrib* e mantida aqui – foi a de **extração literal** das fórmulas e algoritmos do código-fonte de referência do MPAS-Model, em vez de reimplementá-los “do zero” a partir da descrição em artigos ou manuais. A justificativa é dupla: primeiro, garante consistência numérica exata com o que o modelo real produz, eliminando uma classe inteira de divergência que surgiria de decisões de implementação sutilmente diferentes (ordem de operações, escolha entre formas matematicamente equivalentes de uma mesma expressão, tratamento de casos de borda); segundo, torna a origem de cada linha de código rastreável e auditável – cada módulo novo carrega, em comentário de cabeçalho, a referência exata do arquivo/rotina/faixa de linhas do código-fonte original de onde foi extraído, e a versão específica consultada (`mpas-bundle-3.0.2`, confirmada como o fonte exato que gerou os binários de produção usados neste projeto, distinta e por vezes divergente da *branch* principal do repositório público – por exemplo, a fórmula do fator de suavização $s(i)$ na Eq. (6.3) usa z_t como referência de altura na versão 3.0.2, mas z_h na *branch master* mais recente).

11.2 Estrutura de módulos

A Table 11.1 resume os módulos Fortran novos, organizados pela fase a que pertencem, com suas dependências e origem no código-fonte de referência.

Tabela 11.1: Módulos Fortran novos desenvolvidos neste trabalho.

Módulo/programa	Fase	Origem no código-fonte de referência
<code>hinterp_native.F90</code>	1	Cópia estrutural de <code>convert_mpas.F90</code> , reutilizando sem modificação <code>mpas_mesh</code> , <code>target_mesh</code> , <code>remapper</code>
<code>vertical_grid.F90</code>	2	<code>init_atm_case_gfs</code> , <code>config_tc_vertical_grid</code> bloco
<code>vinterp_native.F90</code>	3	Função local <code>vertical_interp</code> de <code>init_atm_case_gfs</code>
<code>hydrostatic.F90</code>	4	<code>init_atm_case_gfs</code> , <code>config_met_interp</code> bloco
<code>surface_fields.F90</code>	6	<code>mpas_atmphys_initialize_real.F</code> , <code>mpas_atmphys_date_time.F</code>
<code>gen_init_native.F90</code>	2+3+4+6	Programa-driver, novo (orquestração + escrita NetCDF)
<code>gen_lbc_native.F90</code>	3+4+7	Programa-driver, novo (reusa grade vertical do <code>init.nc</code>)
<code>namelist_config.F90</code>	10	Novo (leitura de <i>namelist</i> real, sem equivalente direto)

11.3 Convenção de precisão numérica: uma exceção deliberada

Todo o restante do projeto *selfgrib* usa precisão dupla (RKIND, `real*8`) como padrão. A única exceção deliberada é o bloco de suavização iterativa de terreno da Fase 2 (Section 6.4): o log real de produção revela que o binário do `init_atmosphere_model` foi compilado com "Default real precision: single" – isto é, precisão simples (`real*4`) em toda a base de código, não apenas nesse bloco. Como o critério de aceitação por limiar desse algoritmo (Eq. (6.3)) é sensível a diferenças mínimas de arredondamento, compostas ao longo de até 85 passadas por até 53 níveis, as variáveis diretamente envolvidas nessa suavização (terreno, h_x , acumuladores) foram declaradas com um parâmetro de precisão simples dedicado (SPKIND=`selected_real_kind(6)`), propositalmente, para casar com o binário real – uma escolha investigada e confirmada experimentalmente (Achado técnico 6.1) como *não* sendo, sozinha, a causa da divergência numérica original, mas mantida por rigor mesmo após a causa real ter sido encontrada, já que reduz ruído residual de comparação.

11.4 Leitura do *namelist* real em tempo de execução

O módulo `namelist_config.F90` define um tipo derivado (`init_atm_config_type`) que espelha os campos de configuração efetivamente usados pelas fases deste trabalho, com valores-padrão sensatos, e uma rotina (`read_init_atm_namelist`) que lê, via bloco

NAMelist nativo do Fortran, os grupos `&nhyd_model`, `&dimensions`, `&data_sources`, `&vertical_grid`, `&interpolation_control` e `&preproc_stages` do arquivo `namelist.init_atmosphere` real do experimento – o mesmo arquivo já consumido pela rota original, sem nenhum formato novo introduzido. A leitura é tolerante a grupos ou variáveis ausentes de um *namelist* específico (por exemplo, `config_fg_interval` só existe no *namelist* do `lbc_run`, não no do `init_run`): os valores-padrão do tipo derivado são inicializados antes da leitura, de modo que uma variável ausente simplesmente mantém seu padrão em vez de ficar indefinida.

Listing 11.1: Trecho ilustrativo de `namelist_config.F90`: leitura tolerante de múltiplos grupos de *namelist*.

```

1  type :: init_atm_config_type
2      real (kind=RKIND) :: config_ztop = 30000.0_RKIND
3      integer          :: config_nsm = 30
4      real (kind=RKIND) :: config_dzmin = 0.3_RKIND
5      logical          :: config_blend_bdy_terrain = .false.
6      logical          :: config_frac_seaice      = .true.
7      character (len=64) :: config_start_time = ''
8      ! ... demais campos config_*
9  end type init_atm_config_type
10
11 subroutine read_init_atm_namelist(filename, cfg)
12     ! ...
13     open(newunit=unit_nml, file=trim(filename), status='old', action='read')
14     read(unit_nml, nml=nhyd_model, iostat=ios); rewind(unit_nml)
15     read(unit_nml, nml=vertical_grid, iostat=ios); rewind(unit_nml)
16     ! ... demais grupos, cada um tolerante a ausencia
17     close(unit_nml)
18     call parse_config_start_time(cfg)
19 end subroutine

```

Essa generalização, aplicada de forma consistente aos dois programas principais (`gen_init_native`, `gen_lbc_native`), resultou em uma constatação importante de validação: recompilar e re-executar toda a cadeia com os parâmetros agora lidos dinamicamente, em vez de fixos, produziu um resultado **numericamente idêntico, bit a bit**, ao obtido com os valores anteriormente fixos no código-fonte – confirmando que a generalização não alterou nenhum cálculo, apenas a origem dos parâmetros.

11.5 Reúso de dependências e organização do build

O sistema de *build* (**Makefile**) mantém a convenção já estabelecida no projeto *selfgrib* de compilar `convert_mpas` como dependência externa antes do restante da pipeline, já que `hinterp_native` (Fase 1) reutiliza diretamente seus módulos objeto já compilados (`mpas_mesh.o`, `target_mesh.o`, `remapper.o`, `scan_input.o`), sem recompilá-los. Os demais módulos novos (Fases 2 a 7) dependem apenas de `mpas_kind_types.F90` e `mpas_constants.F90`, ambos já existentes no projeto – nenhuma dependência externa nova foi introduzida por este trabalho, além do utilitário NCO (`ncks`) usado na composição final do `init.nc` (Chapter 10), tipicamente já disponível em ambientes de HPC voltados a geociências.

Capítulo 12

Resultados

Este capítulo consolida os resultados quantitativos de validação apresentados ao longo dos capítulos anteriores, organizados por nível de integração: (i) validação isolada da Fase 1 (propriedade de exatidão); (ii) validação campo a campo do `init.nc` completo contra um arquivo real de produção; (iii) validação do `lbc.*.nc`, por auto-consistência e por comparação com a referência real; e (iv) validação funcional, comparando a previsão de 24 horas obtida a partir dos arquivos desta rota com a previsão de referência gerada pela rota original. Todos os números desta seção referem-se ao caso de estudo usado consistentemente em todo o desenvolvimento: malha regional **SouthAmerica** (recorte de $x1.163842$, 17064 células, 51648 arestas, ~ 60 km de resolução nominal), rodada iniciada em 2026-01-01_00:00:00.

12.1 Validação da interpolação horizontal (Fase 1)

Conforme discutido na Section 5.4, a interpolação de uma malha de teste (2562 células) para os próprios centros de célula dessa mesma malha (*auto-scatter*) produziu erro identicamente nulo em 100% das células, tanto para os valores de campo quanto para as coordenadas de saída – confirmando a implementação correta da propriedade de exatidão nos vértices da interpolação baricêntrica (Section 2.3).

12.2 Validação do `init.nc` completo

Tabela 12.1: Erro absoluto campo a campo entre o `init.nc` gerado por esta rota e o `init.nc` real de produção (17064 células, malha `SouthAmerica`, mesmo tempo inicial). Compilado e executado no build/ambiente real de produção (cluster Jaci), não apenas em ambiente de desenvolvimento local.

Campo	Faixa real (mín/máx)	Erro médio	Erro máximo
ρ [kg/m ³]	0,018 / 1,265	$1,5 \times 10^{-4}$	$1,84 \times 10^{-2}$
θ [K]	273,1 / 825,0	0,114	3,56
q_v [kg/kg]	0 / 0,0192	$5,1 \times 10^{-5}$	$2,52 \times 10^{-3}$
u [m/s]	-76 / 77	$8,5 \times 10^{-2}$	4,47
w [m/s]	-1,52 / 1,08	$4,0 \times 10^{-3}$	1,63
<code>surface_pressure</code> [Pa]	$5,76 \times 10^4$ / $1,03 \times 10^5$	5,9	1792
<code>precipw</code> [mm]	0,92 / 62,1	0,31	4,46
<code>skintemp</code> [K]	273,0 / 307,5	0,21	7,07
<code>tmn</code> [K]	266,2 / 302,1	0,15	6,95
<code>vegfra</code> [%]	0 / 90,6	$7,3 \times 10^{-7}$	$1,1 \times 10^{-5}$

Campos exatos ou essencialmente exatos (erro nulo ou de arredondamento de armazenamento, $< 10^{-6}$): `xland`, `xice`, `seaice`, `sh2o`, `dz`, `u_init`, `v_init`, `qv_init`, `t_init`, `qc`, `qr`, `sfc_albbck`, `dzs`, `zs`. O arquivo final contém a totalidade das 135 variáveis do `init.nc` real (mais uma variável diagnóstica auxiliar, `psfc`, sem correspondente no arquivo real e sem uso pelo núcleo do modelo). Os resultados na Table 12.1 foram obtidos compilando e executando o pipeline diretamente no ambiente de produção (compilador GNU 13.3.0, bibliotecas NetCDF/PIO do cluster), reproduzindo, **dígito a dígito**, os mesmos valores já obtidos em ambiente de desenvolvimento local – uma confirmação adicional de portabilidade e robustez numérica da implementação frente a diferenças de compilador e biblioteca.

12.3 Validação do `lbc.*.nc`

A auto-consistência entre o `init.nc` e um `lbc.*.nc` gerado para o *mesmo* tempo inicial (mesma malha vertical, mesmo *first-guess*) foi verificada como **exata**: diferença identicamente nula em ρ , θ , q_v , u e w – o resultado esperado, já que ambos os programas executam exatamente o mesmo cálculo de Fases 3–4 para esse tempo.

Tabela 12.2: Erro absoluto médio/máximo entre o `lbc.*.nc` gerado por esta rota e os arquivos reais de produção, nos cinco tempos de fronteira do experimento (intervalo de 6 h).

Tempo	u [m/s]	ρ	θ [K]	w [m/s]
t_0 (00h)	0,086 / 4,47	0,0056 / 0,043	0,114 / 3,56	0,004 / 1,63
$t_0 + 6h$	0,062 / 5,69	0,0056 / 0,044	0,055 / 2,86	0,004 / 1,59
$t_0 + 12h$	0,055 / 7,10	0,0056 / 0,041	0,051 / 3,15	0,004 / 1,60
$t_0 + 18h$	0,052 / 12,2	0,0057 / 0,042	0,060 / 4,65	0,004 / 1,45
$t_0 + 24h$	0,055 / 12,5	0,0057 / 0,042	0,059 / 2,91	0,004 / 1,46

O erro médio permanece pequeno e estável ao longo dos cinco tempos; o erro máximo pontual cresce moderadamente nos tempos mais tardios – compatível com divergência acumulada de previsão real ao longo de até 24 horas somada ao resíduo de fronteira já conhecido (Chapter 13), não com instabilidade numérica. O único problema identificado nesta comparação não é desta implementação: o campo `lbc_qv` do arquivo de referência real é **constante em todos os 55 níveis verticais por célula** – fisicamente implausível para um campo de umidade específica, e inconsistente com o comportamento correto (variável por nível) dos demais campos do mesmo arquivo –, sugerindo um problema já existente na cadeia de geração de referência (rota via WPS), não atribuível a esta rota.

12.4 Validação funcional: execução do modelo e comparação de previsão

A previsão de 24 horas gerada a partir dos arquivos desta rota (Chapter 10) foi comparada, campo a campo e por tempo, com a previsão de referência (mesma malha, mesmo período, gerada pela rota original via WPS).

Tabela 12.3: Erro absoluto médio/máximo entre a previsão gerada a partir dos arquivos desta rota e a previsão de referência, em três tempos de saída ao longo da integração de 24 h.

Tempo de previsão	θ [K]	ρ	q_v [kg/kg]	u [m/s]	v [m/s]
+0h	0,11 / 3,6	$1,5 \times 10^{-4}$ / $1,8 \times 10^{-2}$	$5,1 \times 10^{-5}$ / $2,5 \times 10^{-3}$	0,08 / 4,5	0,00
+12h	0,27 / 6,4	$9,2 \times 10^{-4}$ / $2,9 \times 10^{-2}$	$4,7 \times 10^{-4}$ / $1,8 \times 10^{-2}$	0,44 / 9,2	0,007
+24h	0,41 / 9,8	$1,1 \times 10^{-3}$ / $3,0 \times 10^{-2}$	$6,4 \times 10^{-4}$ / $1,8 \times 10^{-2}$	0,58 / 12,0	0,010

O padrão observado – erro pequeno na inicialização, crescendo de forma suave e limitada ao longo da integração, sem nenhum salto abrupto ou sinal de instabilidade (ausência total de valores NaN em qualquer campo, em qualquer tempo) – é o esperado para duas trajetórias vizinhas de um sistema dinâmico não linear sensível a condições iniciais (divergência tipo Lyapunov): as duas rotas partem de condições iniciais ligeiramente

diferentes (pelo resíduo residual documentado nos Capítulos 6 e 9) e evoluem, sob a mesma física e a mesma dinâmica, para estados progressivamente mais distantes – não uma divergência causada por um erro estrutural na implementação, que se manifestaria como blowup numérico, quebra de conservação, ou surgimento de valores não físicos, nenhum dos quais foi observado. O próprio log de execução do modelo confirma a sanidade física ao longo de toda a integração: os valores globais de mínimo/máximo de w e u , impressos a cada passo de tempo, permanecem dentro de faixas meteorologicamente plausíveis (por exemplo, $w \in [-0,61, +2,22]$ m/s, $u \in [\pm 65, \pm 77]$ m/s) durante toda a janela de 24 horas.

Capítulo 13

Discussão e limitações conhecidas

Este capítulo consolida, de forma explícita, as simplificações deliberadas e limitações conhecidas da implementação atual – distintas de bugs (já corrigidos ao longo dos capítulos anteriores), no sentido de que são escolhas de escopo documentadas, com risco avaliado e aceito para o caso de estudo validado, mas que podem se tornar relevantes em outros contextos de uso.

13.1 Mistura de terreno de fronteira não implementada

A limitação de maior impacto numérico mensurável identificada é a ausência do passo `config_blend_bdy_terrain`: no algoritmo de referência, antes de gerar a grade vertical (Chapter 6), a altura de terreno das células do anel de fronteira da malha regional (`bdyMaskCell ≥ 1`) é misturada com o terreno do dado de *fundo* (*first-guess*), suavizando a transição entre o domínio regional e a condição de contorno que o alimentará. Sem esse passo, o resíduo de ~ 171 m documentado na Section 6.6 permanece concentrado exatamente nessa faixa de células – que é, por definição, onde a fronteira lateral atua. Para o caso de validação usado neste relatório, esse resíduo não impediu uma integração de 24 horas fisicamente sã (Chapter 12), mas é a lacuna mais provável de precisar de atenção em usos futuros que dependam de maior precisão perto da borda do domínio regional, ou em domínios com relevo mais acentuado próximo à fronteira do que o caso testado.

13.2 Reamostragem vertical de solo simplificada

O algoritmo de referência reamostra os perfis de temperatura e umidade de solo do *first-guess* para as quatro profundidades-padrão do esquema Noah usando uma interpolação linear por profundidade própria (distinta e independente da grade vertical atmosférica). Esta implementação copia esses campos diretamente do *first-guess* sem essa reamostragem

– uma simplificação segura apenas porque, no caso de uso deste projeto, tanto a origem quanto o destino do *first-guess* são o próprio MPAS-A, com o mesmo esquema Noah e as mesmas quatro profundidades de solo. Essa simplificação deixa de ser válida se o *first-guess* vier de uma fonte com profundidades de solo diferentes (por exemplo, GFS via WPS/ungrib).

13.3 Reclassificação de gelo marinho não implementada

O algoritmo de referência reclassifica células de água muito fria (temperatura de pele abaixo de um limiar configurável, `config_tsk_seaice_threshold`) como gelo marinho ou terra, ajustando em cascata o tipo de uso do solo, textura de solo, albedo máximo de neve, e os perfis de temperatura/umidade de solo dessas células. Essa reclassificação não foi implementada. A omissão é irrelevante para o caso de validação (América do Sul: o campo `SEAICE` do *first-guess* é identicamente zero em toda a malha, confirmado diretamente contra o dado real), mas se tornaria relevante em qualquer malha regional que inclua regiões de alta latitude com gelo marinho real.

13.4 Umidade específica via umidade relativa não implementada

O código de referência oferece dois caminhos para diagnosticar a razão de mistura de vapor d'água: a partir da umidade específica do *first-guess* (`config_use_spechumd=.true.`) ou, na ausência desse campo, a partir da umidade relativa via a função de saturação de Flatau/Thompson (`rs1f`). Apenas o primeiro caminho foi implementado – suficiente porque todos os casos de estudo deste projeto, até o momento, usam `config_use_spechumd=.true.` com um *first-guess* que de fato contém esse campo. O programa detecta explicitamente a configuração oposta e encerra com uma mensagem de erro clara, em vez de produzir um resultado silenciosamente incorreto.

13.5 Escopo de validação

Todo o processo de validação numérica e funcional apresentado neste relatório usa uma única malha regional (`SouthAmerica`, ~ 60 km), um único caso de estudo (24 horas de integração a partir de 2026-01-01_00:00:00), e $n_{VertLevels} = 55$. A lista de 61 níveis de pressão fixa usada como representação intermediária entre a extração de campos e a interpolação horizontal nativa (`pressure_levels.F90`, herdada sem modificação da rota

original) é derivada de medianas estatísticas de altura de nível de modelo – não específica da América do Sul, e portanto esperada como razoável em outras malhas – mas nunca de fato testada em uma malha/resolução distinta. A generalização estrutural do código (Section 4.4) remove a maior barreira à reprodução em outros casos, mas não substitui uma validação explícita nesse cenário mais amplo.

13.6 Um interpolante linear por partes: limite de suavidade

A interpolação baricêntrica adotada (Section 2.3) é C^0 – contínua entre triângulos adjacentes do dual de Delaunay, mas com derivada descontínua nas arestas desses triângulos. Para o problema resolvido aqui (gerar uma condição inicial/de fronteira, consumida integralmente pelo núcleo dinâmico do modelo, que já aplica sua própria suavização/filtro numérico durante a integração), essa limitação não se manifestou como problema na validação funcional (Chapter 12). Ela poderia se tornar relevante caso o mesmo motor de interpolação fosse reaproveitado para outro uso que dependa diretamente de gradientes horizontais suaves do campo interpolado – caso em que os interpolantes de maior continuidade discutidos na revisão de literatura (Chapter 3, Hiyoshi e Sugihara [8]) seriam um candidato natural de extensão.

13.7 Síntese das limitações

Tabela 13.1: Síntese das simplificações/limitações conhecidas, seu impacto no caso validado, e a condição sob a qual se tornariam relevantes.

Limitação	Impacto no caso validado	Torna-se relevante quando
Sem mistura de terreno de fronteira	Resíduo de ~ 171 m concentrado no anel de fronteira	Precisão perto da borda regional importa; relevo acentuado na fronteira
Solo copiado sem reamostragem vertical	Nenhum (mesma malha/esquema na origem e destino)	<i>First-guess</i> com esquema de solo/profundidades distintas
Sem reclassificação de gelo marinho	Nenhum (sem gelo marinho no caso)	Malha regional em alta latitude
Sem q_v via RH	Nenhum	<i>First-guess</i> sem umidade específica direta
	(<code>config_use_specumd</code> sempre <code>.true.</code> nos casos testados)	
Validação em 1 malha/1 caso	–	Uso em outra resolução/domínio/estação do ano
Interpolação C^0 (baricêntrica linear)	Nenhum observado	Uso que dependa de gradiente horizontal suave do campo interpolado

Capítulo 14

Conclusão

14.1 Síntese

Este trabalho demonstrou que é possível eliminar inteiramente a etapa de reamostragem em grade latitude-longitude regular do processo de geração de condição inicial e de fronteira lateral do MPAS-A, interpolando diretamente entre a malha de Voronoi nativa de origem (uma rodada global já concluída) e a malha de Voronoi nativa de destino (o domínio regional), e escrevendo os arquivos `init.nc/lbc/*.nc` consumidos pelo núcleo dinâmico `mpas_atmosphere` sem depender do `init_atmosphere_model` original. A viabilidade dessa rota não era óbvia *a priori*: a investigação do código-fonte de referência mostrou que o formato binário intermediário do WPS é, por definição, uma grade regular, de modo que nenhum “escopo mínimo” preservaria o ganho de precisão da interpolação nativa sem substituir inteiramente o caminho – horizontal, vertical, hidrostático e de superfície – por implementação própria.

O método central – interpolação baricêntrica na malha dual de Delaunay – não é uma proposta original deste trabalho, mas a aplicação de uma técnica já validada pela comunidade (via o sistema de assimilação MPAS-DART) a um novo contexto de uso. A contribuição prática está na extração cuidadosa, tradução literal e validação exaustiva de cada fórmula fisicamente necessária do código-fonte de referência do MPAS-Model – um processo de engenharia reversa disciplinado que revelou, ao longo do caminho, tanto suposições incorretas do planejamento inicial quanto bugs reais de implementação, todos documentados com sua investigação e correção nos capítulos técnicos deste relatório.

Um padrão metodológico se repete em praticamente todos os problemas mais difíceis encontrados: a causa raiz raramente estava onde o sintoma aparecia. A divergência de ~ 2 km na grade vertical não estava no algoritmo de suavização (onde o sintoma se manifestava), mas em uma suposição incorreta sobre um valor padrão de configuração calculado dezenas de linhas antes. Os dois bugs que impediam a execução real do modelo não eram de física nem de numérica – eram de formato de uma única variável

de metadado (`xtime`) inicialmente considerada secundária. Em ambos os casos, uma validação aparentemente completa (revisão de código linha a linha; comparação numérica campo a campo) não foi suficiente até que a validação fosse estendida às dependências reais do problema – o código *upstream* do trecho suspeito, no primeiro caso; o consumidor final de verdade, no segundo.

14.2 Estado atual e alcance da validação

Ao final deste trabalho, a rota nativa *voronoi-to-voronoi* está validada em duas camadas independentes: numericamente, campo a campo, contra arquivos `init.nc/lbc.*.nc` reais de produção; e funcionalmente, executando o `mpas_atmosphere` real a partir desses arquivos e obtendo uma previsão de 24 horas fisicamente sã, cuja divergência frente à rota de referência cresce de forma suave e limitada, consistente com o comportamento esperado de duas trajetórias vizinhas de um sistema dinâmico caótico – não com um artefato de implementação. Essa validação, no entanto, foi conduzida em um único par malha/caso de estudo (Chapter 13); a rota foi generalizada estruturalmente (leitura de *namelist* real em tempo de execução) para não depender de recompilação em outros cenários, mas ainda não foi exercitada neles.

14.3 Trabalhos futuros

Três direções concretas de continuidade emergem diretamente das limitações documentadas no Chapter 13:

1. **Implementar a mistura de terreno de fronteira** (`config_blend_bdy_terrain`), fechando o resíduo residual de ~ 171 m concentrado no anel de células de fronteira – a lacuna de maior impacto numérico mensurável remanescente, e a mais diretamente relevante caso a fronteira lateral se torne um ponto crítico de precisão em usos futuros.
2. **Ampliar a validação para outras malhas, resoluções e estações do ano**, exercitando a generalização estrutural já implementada (Section 4.4) e verificando se os 61 níveis de pressão fixa intermediários – derivados estatisticamente de uma única malha/caso – permanecem adequados em contextos com perfil vertical de pressão substancialmente distinto.
3. **Avaliar interpolantes de maior continuidade** sobre a malha dual de Delaunay (por exemplo, a família de interpolantes generalizados de Sibson/Laplace descrita por Hiyoshi e Sugihara [8]) como substituto opcional da interpolação baricêntrica linear, caso alguma aplicação futura do mesmo motor de remapeamento – por exemplo, fora

do contexto de condição inicial/fronteira, em algum uso que dependa diretamente de gradiente horizontal suave – se mostre sensível à descontinuidade de derivada nas arestas dos triângulos de Delaunay.

Adicionalmente, uma investigação independente e de menor prioridade identificada ao longo deste trabalho, mas fora de seu escopo direto: o campo `lbc_qv` dos arquivos de referência reais de produção mostrou-se constante em todos os níveis verticais por célula (Chapter 12) – um comportamento fisicamente implausível que sugere um problema já existente na cadeia de geração da rota original (via WPS), não introduzido por este trabalho, e que mereceria investigação própria caso a fronteira lateral de referência continue sendo usada como padrão de comparação em trabalhos futuros.

Referências

- [1] Ahmed Abdelkader et al. “VoroCrust: Voronoi Meshing Without Clipping”. Em: *arXiv preprint arXiv:1902.08767* (2023).
- [2] Stefano Bondù et al. “3D Voronoi grid dedicated software for modeling gas migration in deep layered sedimentary formations with TOUGH2-TMGAS”. Em: *Computers & Geosciences* 108 (2017), pp. 50–55. DOI: [10.1016/j.cageo.2017.02.001](https://doi.org/10.1016/j.cageo.2017.02.001).
- [3] P. Camps, M. Baes e W. Saftly. “Using 3D Voronoi grids in radiative transfer simulations”. Em: *Astronomy & Astrophysics* 560 (2013), A35. DOI: [10.1051/0004-6361/201322281](https://doi.org/10.1051/0004-6361/201322281).
- [4] Qiang Du e Max Gunzburger. “Grid generation and optimization based on centroidal Voronoi tessellations”. Em: *Applied Mathematics and Computation* 133.2–3 (2002), pp. 591–607. DOI: [10.1016/S0096-3003\(01\)00260-0](https://doi.org/10.1016/S0096-3003(01)00260-0).
- [5] Michael G. Duda. *Examining the Performance and Scalability of MPAS-Atmosphere*. GungHo Meeting presentation. Apresentação técnica, 30 abril–1 maio de 2014. National Center for Atmospheric Research, 2014.
- [6] Darren Engwirda. “JIGSAW-GEO (1.0): locally orthogonal staggered unstructured grid generation for general circulation modelling on the sphere”. Em: *Geoscientific Model Development* 10 (2017), pp. 2117–2140. DOI: [10.5194/gmd-10-2117-2017](https://doi.org/10.5194/gmd-10-2117-2017).
- [7] Soyoung Ha, Chris Snyder, William C. Skamarock, Jeffrey Anderson e Nancy Collins. “Ensemble Kalman Filter Data Assimilation for the Model for Prediction Across Scales (MPAS)”. Em: *Monthly Weather Review* 145.11 (2017), pp. 4673–4692. DOI: [10.1175/MWR-D-17-0145.1](https://doi.org/10.1175/MWR-D-17-0145.1).
- [8] Hisamoto Hiyoshi e Kokichi Sugihara. “Voronoi-Based Interpolation with Higher Continuity”. Em: *Proceedings of the 16th Annual Symposium on Computational Geometry*. 2000, pp. 242–250. DOI: [10.1145/336154.336210](https://doi.org/10.1145/336154.336210).
- [9] Joseph B. Klemp. “A Terrain-Following Coordinate with Smoothed Coordinate Surfaces”. Em: *Monthly Weather Review* 139.7 (2011), pp. 2163–2169. DOI: [10.1175/MWR-D-10-05046.1](https://doi.org/10.1175/MWR-D-10-05046.1).
- [10] Luciano Melodia e Richard Lenz. “Persistent Homology as Stopping-Criterion for Voronoi Interpolation”. Em: *arXiv preprint arXiv:1911.02922* (2026).

- [11] MPAS-JEDI Development Team. *Static B in MPAS-JEDI*. Apresentação técnica. 17 de novembro de 2021. NCAR/UCAR – Joint Effort for Data assimilation Integration, 2021.
- [12] Maya A. Petkova, Guillaume Laibe e Ian A. Bonnell. “Fast and accurate Voronoi density gridding from Lagrangian hydrodynamics data”. Em: *arXiv preprint arXiv:1710.07108* (2017).
- [13] Luan F. Santos e Pedro S. Peixoto. “Topography-based local spherical Voronoi grid refinement on classical and moist shallow-water finite-volume models”. Em: *Geoscientific Model Development* 14 (2021), pp. 6919–6944. DOI: [10.5194/gmd-14-6919-2021](https://doi.org/10.5194/gmd-14-6919-2021).
- [14] William C. Skamarock, Joseph B. Klemp, Michael G. Duda, Laura D. Fowler, Sang-Hun Park e Todd D. Ringler. “A Multiscale Nonhydrostatic Atmospheric Model Using Centroidal Voronoi Tessellations and C-Grid Staggering”. Em: *Monthly Weather Review* 140.9 (2012). Descrição de referência do núcleo dinâmico MPAS-A; PDF local: [76424-global-nonhydrostatic-modelling-using-voronoi-meshes_0.pdf](#) (versão de conferência, Skamarock et al., “Global Nonhydrostatic Modeling Using Voronoi Meshes”), pp. 3090–3105. DOI: [10.1175/MWR-D-11-00215.1](https://doi.org/10.1175/MWR-D-11-00215.1).
- [15] Huanhuan Yang, Max Gunzburger e Lili Ju. “Fast Spherical Centroidal Voronoi Mesh Generation: A Lloyd-preconditioned LBFGS Method in Parallel”. Em: *arXiv preprint arXiv:1709.06924* (2018).